



Birzeit University
Faculty of Engineering & Technology
Department of Electrical & Computer Engineering

ENCS5300, GRADUATION PROJECT

Smart RFID Inventory Tracking System

Prepared By:
Kareem Alqutob - 1211756
Ahmad Elayyan - 1210443
Husain Abugosh - 1210338

Section 26

Supervised By:
Dr. Nofal Nofal

A Graduation Project submitted to the Department of Electrical and
Computer Engineering in partial fulfillment of the requirements for the
degree of B.Sc. in Computer Engineering

Birzeit
July, 2026

Abstract

This project presents a smart, Radio Frequency Identification (RFID)-driven inventory management system designed to address persistent challenges in laboratories and warehouses, where traditional manual and barcode-based processes often lead to human error, outdated records, and inconsistencies between physical assets and digital databases. The core of the system leverages RFID technology to enable real-time detection, tracking, location awareness, and automatic quantity updates for tagged items, significantly reducing dependency on manual labor and improving inventory accuracy.

To extend system functionality beyond the limitations of RFID specifically its inability to identify untagged or newly introduced objects a complementary Computer Vision (CV) module is integrated into a mobile application, allowing visual recognition and classification of items through the device's camera. This dual-capability design ensures full coverage: RFID maintains continuous automated tracking and localization of known inventory, while the vision subsystem assists in registering unfamiliar items by suggesting new database entries.

System evaluation demonstrates substantial improvements in accuracy, update reliability, item traceability, and synchronization between physical and digital inventory states. Overall, this project provides a scalable and autonomous framework for modern inventory management, adaptable to diverse operational environments such as laboratories, warehouses, and organizations.

المستخلص

يقدم هذا المشروع نظاما ذكيا لإدارة المخزون يعتمد بشكل رئيسي على تقنية التعرف بالترددات الراديوية (RFID) ، وذلك لمعالجة التحديات المتكررة في المختبرات والمستودعات، حيث تؤدي الأساليب التقليدية، سواء اليدوية أو المعتمدة على الباركود، إلى أخطاء بشرية وتحديثات غير دقيقة وتباين بين الموجودات الفعلية وقواعد البيانات الرقمية. يعتمد النظام على تقنية RFID لتنفيذ الكشف اللحظي عن العناصر المعرفة، وتتبعها، وتحديد مواقعها داخل المستودع، وتحديث كمياتها تلقائيا، مما يقلل بشكل كبير من التدخل البشري ويرفع موثوقية بيانات المخزون.

ولمعالجة القصور في تقنية RFID خصوصا عدم قدرتها على التعرف على العناصر غير الموسومة أو المضافة حديثا تم دمج وحدة رؤية حاسوبية ضمن تطبيق الهاتف المحمول، تتيح التعرف البصري على العناصر وتصنيفها باستخدام كاميرا الجهاز.

يوفر هذا التكامل تغطية شاملة لإدارة المخزون، حيث تتولى تقنية RFID عملية التتبع الآلي وتحديد مواقع العناصر المعروفة، بينما تساعد وحدة الرؤية في تسجيل العناصر الجديدة عند اكتشافها لأول مرة. وتُظهر نتائج التقييم تحسنا واضحا في دقة التتبع، وموثوقية التحديثات، وسهولة الوصول إلى العناصر، والتزامن بين الواقع الفعلي وسجلات المخزون. وبذلك يقدم المشروع إطارا قابلاً للتوسع وذاتي العمل لإدارة المخزون في البيئات التشغيلية المختلفة داخل المختبرات والمستودعات والمؤسسات.

Table of Contents

English Abstract	I
Arabic Abstract	II
Table of Contents	III
List of Tables	VII
List of Figures	VIII
List of Abbreviations	IX
1 Introduction and Motivation	1
1.1 Motivation	2
1.2 Problem Statement	2
1.3 Methodology	3
1.4 Contribution	3
1.5 Report Outline	3
2 Background	5
2.1 RFID Theory	6
2.2 RFID Tags and Operating Principles	6
2.2.1 Internal Structure and Operation of Passive RFID Tags	6
2.2.2 Tag Identifier and Memory Structure (EPC Gen2 Overview)	7
2.3 RFID Frequency Bands and Communication Model	8
2.4 Theoretical Limitations of RFID-Based Tracking	8
2.4.1 RFID-Based Item Allocation Concept	9
2.5 Computer Vision as a Data Entry and Verification Tool	9
2.6 RFID System Deployment in Inventory Environments	9

3	Related Work	10
3.1	Case Study 1: Integration of RFID and CV for Autonomous Inventory Management	11
3.2	Case Study 2: Airport Smart Inventory and Baggage Management Using RFID	11
3.3	Case Study 3: Amazon Smart Inventory Management Using RFID and Amazon Web Services (AWS)	11
3.4	Research Study : Zone-Based RFID Tracking Approaches	12
3.5	Research Study : RFID-Based Asset Management in Controlled Environments	12
3.6	Comparative Analysis of Reviewed Systems	12
4	System Design and Architecture	15
4.1	Overall System Architecture	16
4.2	System Layered Architecture	17
4.3	RFID Deployment Strategy and Range Control	19
4.3.1	Zone Readers vs. Door (In/Out) Reader	19
4.3.2	Read Range Control Methods	20
4.3.3	Approximate Coverage Estimation	20
4.3.4	Key Analytical Models (Supporting Equations)	20
4.3.5	RFID Interference and Mitigation Strategies	21
4.4	Hardware Components	22
4.4.1	Passive Ultra-High Frequency (UHF) RFID Tags	22
4.4.2	UHF RFID Reader Module	22
4.4.3	Raspberry Pi 4 (4GB Random Access Memory (RAM))	22
4.4.4	Mobile Device	23
4.5	Software Components	23
4.5.1	Technology Selection Rationale	24
4.6	System Requirements	25
4.6.1	Functional Requirements	25
4.6.2	Non-Functional Requirements	26
4.7	System Workflow	26
4.7.1	Item Entry into Warehouse	26
4.7.2	Zone Allocation and Tracking	27
4.7.3	New Item Registration	27
4.7.4	Item Exit from Warehouse	27

4.7.5	System Synchronization	28
5	System Implementation	30
5.1	Edge layer Implementation:	32
5.1.1	Raspberry Pi OS Installation and Initial Setup	32
5.1.2	Universal Asynchronous Receiver-Transmitter (UART) Configura- tion on Raspberry Pi	33
5.1.3	Single RFID Reader Integration	33
5.1.4	Multi-Reader Setup Using Universal Serial Bus (USB)-to-UART Adapter	34
5.1.5	RFID Data Collection Using Python	36
5.1.6	Received Signal Strength Indicator (RSSI) Extraction and Closest Reader Detection	36
5.1.7	Handling Reader Interference	37
5.1.8	Physical Testing with Different Materials	38
5.1.9	Preparing the JSON Payload	38
5.1.10	Sending Data from Raspberry Pi to Server	39
5.2	Backend Layer	39
5.2.1	Backend Architecture	39
5.2.2	Database Schema Design	41
5.2.3	API Design	42
5.2.4	RFID Detection Processing	43
5.2.5	Inventory and Order Management	46
5.2.6	Real-Time Communication	47
5.2.7	Mobile Application API	48
5.2.8	Error Handling and Validation	49
5.3	Dashboard Frontend Layer	50
5.3.1	Website	50
5.3.2	Mobile App	52
5.4	CV Integration Layer	54
5.5	Full System Integration and Validation	56
5.5.1	Full System Story Scenario	57
5.5.2	End-to-End Workflow Testing	58
5.5.3	RFID Scenario Testing	58
5.5.4	Cross-Layer Integration Verification	59
5.5.5	Validation Results	59

6	Evaluation, Conclusion and Future Work	61
6.1	System Evaluation	62
6.2	Limitations and Issues	63
6.3	Conclusion	64
6.4	Future Work	65
	Bibliography	66

List of Tables

3.1	Comparison between reviewed inventory management systems and the proposed system	14
4.1	Technology selection summary (cost, range, power, and development time).	24
4.2	Functional Requirements	25
4.3	Non-Functional Requirements	26
5.1	Comparison of possible multi-reader connection options.	35
5.2	JSON payload fields sent by the Raspberry Pi.	39
5.3	Backend layered architecture.	40
5.4	Main backend database tables.	41
5.5	Main backend API groups.	43
5.6	RFID detection processing cases.	45
5.7	Order status based on physical approval progress.	47
5.8	Main mobile application API endpoints.	48
5.9	Common backend error responses.	49
5.10	Frontend sidebar navigation tabs.	50
5.11	Mobile application roles and available functions.	53
5.12	Main Flutter mobile application screens.	53
5.13	RFID scenario testing cases.	59
5.14	Cross-layer integration verification.	59

List of Figures

2.1	Core components of an RFID-based inventory tracking system	6
2.2	Simplified internal structure of a passive RFID tag (antenna + chip)	7
2.3	Classification of RFID tags based on power source	8
4.1	Overall system architecture of the proposed smart inventory management system.	17
4.2	Layered architecture of the proposed system showing logical separation of system responsibilities.	18
4.3	Operational workflow of the proposed RFID-based inventory system. . . .	29
5.1	Raspberry Pi multi-reader connection setup.	34
5.2	Backend database ER diagram.	42
5.3	RFID detection processing flow.	46
5.4	Warehouse view in the dashboard frontend.	52
5.5	Main pages of the SmartRFID mobile application.	54
5.6	CV processing flow in the SmartRFID inventory system.	56
6.1	System evaluation results for the SmartRFID prototype.	63

List of Abbreviations

RFID	Radio Frequency Identification
UHF	Ultra-High Frequency
HF	High Frequency
LF	Low Frequency
EPC	Electronic Product Code
TID	Tag Identifier
AIDC	Automatic Identification and Data Capture
API	Application Programming Interface
AI	Artificial Intelligence
CV	Computer Vision
AWS	Amazon Web Services
USB	Universal Serial Bus
UART	Universal Asynchronous Receiver-Transmitter
TTL	Transistor-Transistor Logic
GPIO	General Purpose Input/Output
HKIA	Hong Kong International Airport
RAM	Random Access Memory
RSSI	Received Signal Strength Indicator
UML	Unified Modeling Language

Chapter 1

Introduction and Motivation

Contents

1.1	Motivation	2
1.2	Problem Statement	2
1.3	Methodology	3
1.4	Contribution	3
1.5	Report Outline	3

1.1 Motivation

Inventory accuracy is essential for the smooth operation of laboratories, warehouses, and technical facilities, yet many organizations still depend on manual entry or barcode scanning to track their assets [1]. These methods require continuous human involvement and often struggle to keep pace with growing inventories, leading to outdated records, misplaced items, increased search time, and higher operational cost.

RFID technology offers a significant improvement by enabling fast, automated detection of tagged items [2], supporting real-time inventory tracking, automatic quantity updates, and continuous visibility of item movement. Additionally, RFID can be leveraged to provide real-time allocation information, helping users determine where items are stored within the warehouse. However, RFID alone cannot support the registration of untagged or newly introduced items, which still requires manual data entry.

This project is motivated by the need for a scalable and intelligent inventory management solution that preserves RFID as the primary technology for tracking, counting, and real-time allocation, while reducing manual effort during item registration.

To address this limitation, the system integrates a complementary CV-based data entry feature that assists users during the registration of new inventory items without replacing or interfering with RFID-based operations.

1.2 Problem Statement

Many laboratories, warehouses, and organizations still rely on manual or barcode-based inventory systems, which are prone to human error, inefficient tracking, and outdated data [1]. This often results in discrepancies between the physical inventory (actual items in storage) and the digital database records, as well as difficulty in locating items within large or dynamic storage environments.

While RFID technology significantly improves automation by detecting tagged items and supporting real-time allocation, it cannot identify untagged or newly introduced objects, nor can it automatically generate descriptive information for new inventory entries [3]. As a result, item registration remains a manual and error-prone process.

To address these challenges, this project proposes a smart inventory system that combines RFID-based tracking and real-time allocation with a CV-assisted data entry mechanism. When a new item appears, the user captures an image using the mobile application, and the system generates descriptive attributes that are compared against existing database records.

- If a matching item is found, the user is prompted to confirm the match, and the inventory quantity and allocation information are automatically updated based on detected RFID tags.
- If no match is found, the system suggests creating a new inventory entry using the generated description, which is added only after user confirmation.

This approach ensures accurate item registration while maintaining RFID as the sole mechanism for inventory tracking, quantity management, and real-time allocation.

1.3 Methodology

The methodology for developing the inventory management system consisted of four main stages:

1. **Designing the RFID-based tracking and allocation workflow:** This included configuring RFID readers, mapping Electronic Product Code (EPC) codes to database records, enabling real-time quantity updates, and associating items with logical or physical storage locations within the warehouse.
2. **Developing a mobile application for warehouse staff:** The mobile application enables workers to view inventory quantities and real-time location information obtained from RFID readings, and to register new items using a camera-based CV feature that assists with data entry and user confirmation.
3. **Building a web management system for administrators:** This platform provides managers with tools to monitor inventory status, item allocation, and stock movement, manage users, and oversee system operations through a centralized interface.
4. **Integrating and validating system components:** The system was evaluated in a controlled environment, focusing on RFID read reliability, correctness of real-time allocation data, accuracy of CV-assisted item registration, and database consistency.

1.4 Contribution

This project introduces several key contributions:

- **An RFID-centric inventory management system** that provides accurate real-time tracking, automatic quantity updates, and real-time allocation of items within the warehouse.
- **A CV-assisted data entry mechanism** that supports the identification and registration of new inventory items.
- **A centralized backend architecture** that synchronizes inventory data, including quantities and allocation information, between mobile and web platforms in real time.
- **Improved operational efficiency and data accuracy** through automated RFID-based tracking, reduced manual intervention, enhanced item traceability, and real-time visibility of inventory status.

1.5 Report Outline

This report is organized into six chapters, as follows.

Chapter 1 introduces the project motivation, problem statement, methodology, main contributions, and overall report structure. It explains why a smart inventory system is

needed and how the proposed solution combines RFID-based tracking with CV-assisted item registration.

Chapter 2 presents the background and fundamental concepts related to RFID technology, RFID tags and readers, computer vision, backend systems, databases, and inventory management. It provides the theoretical foundation required to understand the proposed system.

Chapter 3 reviews related work and existing RFID-based inventory and asset management systems. It compares industrial and academic approaches, highlights their strengths and limitations, and identifies the gap addressed by the SmartRFID Inventory System.

Chapter 4 describes the system design and architecture. It explains the layered architecture, RFID deployment strategy, hardware and software components, database design, system requirements, operational workflows, and interaction between the edge, backend, dashboard, mobile application, and CV modules.

Chapter 5 presents the system implementation. It describes the Raspberry Pi edge layer, UART and multi-reader setup, RFID data collection, RSSI-based closest-reader logic, backend API implementation, database integration, dashboard frontend, mobile application, CV integration layer, and full system integration and validation.

Chapter 6 presents the system evaluation, limitations and issues, conclusion, and future work. It summarizes the quantitative evaluation results, discusses practical limitations such as RFID interference and backend load, concludes the project outcomes, and suggests improvements for future versions of the system.

Chapter 2

Background

Contents

2.1	RFID Theory	6
2.2	RFID Tags and Operating Principles	6
2.2.1	Internal Structure and Operation of Passive RFID Tags	6
2.2.2	Tag Identifier and Memory Structure (EPC Gen2 Overview) . .	7
2.3	RFID Frequency Bands and Communication Model	8
2.4	Theoretical Limitations of RFID-Based Tracking	8
2.4.1	RFID-Based Item Allocation Concept	9
2.5	Computer Vision as a Data Entry and Verification Tool . . .	9
2.6	RFID System Deployment in Inventory Environments	9

2.1 RFID Theory

RFID is an automatic identification and data capture (Automatic Identification and Data Capture (AIDC)) technology that enables wireless identification of objects through radio-frequency communication [2]. In inventory management systems, RFID serves as the primary theoretical mechanism for tracking and identifying assets by associating each physical item with a unique digital identity.

An RFID system consists of three essential components: RFID tags, RFID readers, and a backend information system [2]. Each tag stores a unique identifier, commonly referred to as an EPC, which allows the system to uniquely identify items and maintain accurate inventory records.

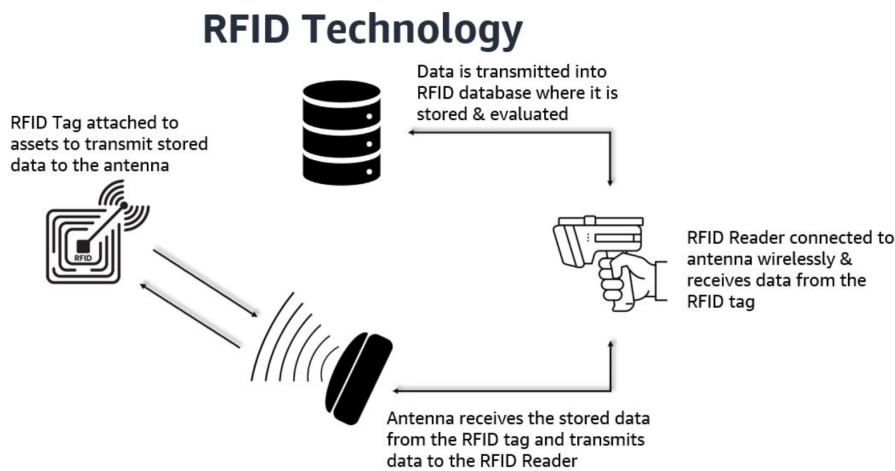


Figure 2.1: Core components of an RFID-based inventory tracking system

2.2 RFID Tags and Operating Principles

RFID tags operate by modulating and reflecting radio-frequency signals transmitted by an RFID reader. Passive RFID tags, which are widely used in inventory applications, do not contain an internal power source and instead harvest energy from the reader's electromagnetic field [4].

When a passive tag enters the reader's field, the induced current powers the tag's integrated circuit, enabling it to transmit its stored EPC back to the reader through backscatter modulation. This theoretical communication model enables identification of a large number of tags with minimal power consumption [5].

RFID tags are classified as passive, semi-passive, or active based on their power source. For large-scale inventory systems, passive RFID tags are generally preferred due to their low cost, compact size, and maintenance-free operation.

2.2.1 Internal Structure and Operation of Passive RFID Tags

A passive RFID tag is primarily composed of an antenna and an integrated circuit (chip). The antenna captures the electromagnetic energy transmitted by the reader and also serves as the interface for communication.

Inside the chip, a rectifier and power-management circuitry convert the received radio-frequency energy into a small DC supply voltage that powers the tag’s digital logic and memory. After powering up, the chip responds by modulating the load presented to the antenna, which causes a controlled change in the reflected signal observed at the reader. This mechanism is known as backscatter modulation and enables the tag to communicate without an internal battery [4, 5].

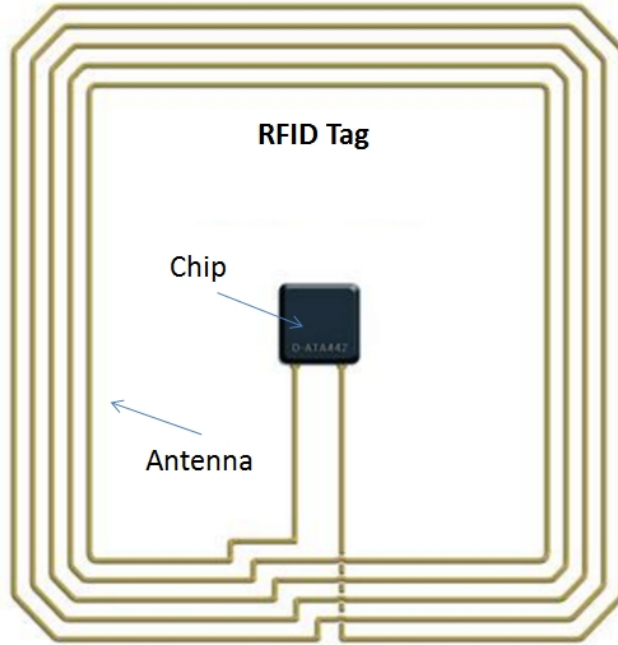


Figure 2.2: Simplified internal structure of a passive RFID tag (antenna + chip)

2.2.2 Tag Identifier and Memory Structure (EPC Gen2 Overview)

In many UHF RFID applications, the most important information stored on a passive tag is its unique identifier, typically encoded in the EPC memory bank. The EPC is often treated as a primary key that can be mapped to richer item metadata in an external information system, rather than storing all descriptive details directly on the tag [2].

Modern UHF tags commonly follow the EPCglobal Class-1 Gen-2 / ISO 18000-63 model, which organizes memory into separate banks for reserved access/kill passwords, EPC data, a manufacturer-provided Tag Identifier (TID), and optional user memory [6]. This separation allows RFID deployments to balance low-cost tag identification with more detailed data management in external software systems.

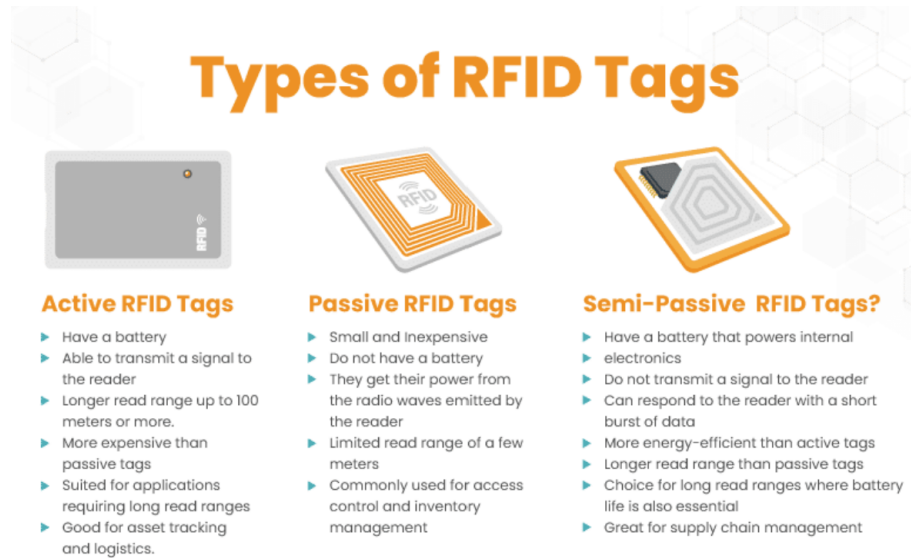


Figure 2.3: Classification of RFID tags based on power source

2.3 RFID Frequency Bands and Communication Model

RFID systems operate across standardized frequency bands, each with distinct theoretical characteristics [6]. Low Frequency (LF) systems operate in the 125–134 kHz range and offer short reading distances with high tolerance to interference. High Frequency (HF) systems operate at 13.56 MHz and are commonly used in access control and smart card applications.

UHF RFID systems operate in the 860–960 MHz range and are widely adopted for inventory tracking due to their extended reading range and fast data transfer rates. UHF systems employ anti-collision protocols that enable simultaneous identification of multiple tags within the reader’s field [7].

Environmental factors such as metallic surfaces, liquids, and electromagnetic interference can affect signal propagation and reading reliability and must be considered during system design.

2.4 Theoretical Limitations of RFID-Based Tracking

Despite its advantages, RFID-based tracking systems have theoretical and practical limitations. RFID technology requires physical tagging of items, which prevents automatic identification of untagged or newly introduced objects [3]. Furthermore, RFID provides identification data but does not offer visual confirmation regarding the physical appearance, placement, or condition of items.

As a result, inventory systems relying solely on RFID may still require complementary mechanisms to assist in item registration and data validation, particularly in dynamic environments.

2.4.1 RFID-Based Item Allocation Concept

In inventory management environments, RFID readings can be used to associate items with logical storage zones or reader locations [8]. By monitoring which reader detects a tag, the system can infer the approximate location of an item within the warehouse without requiring precise positioning coordinates [9]. This zone-based allocation approach enables real-time item allocation and improves item traceability while maintaining low system complexity and cost [10].

2.5 Computer Vision as a Data Entry and Verification Tool

CV is a field of artificial intelligence concerned with enabling machines to interpret visual information from images and video streams [11]. In inventory-related applications, CV can serve as a supporting tool for data entry, item verification, and visual inspection.

Using camera-equipped devices, CV techniques such as object detection and classification can assist users in identifying items, recognizing categories, and extracting descriptive information. These capabilities can facilitate semi-automatic data entry and reduce human error in environments where visual confirmation is useful [12].

From a theoretical perspective, CV can complement RFID by adding visual context to identification workflows. While RFID provides reliable tag-based identity data, CV can support recognition and verification tasks that require visual information.

A common approach for using CV in inventory registration is to convert item images into numerical feature representations, often called image embeddings or feature vectors. These embeddings summarize important visual characteristics of the item, such as shape, color distribution, texture, and other image patterns.

2.6 RFID System Deployment in Inventory Environments

In practical inventory management systems, RFID readers are typically deployed in fixed positions to monitor specific zones or transition points within a facility. Readers may be installed at entry and exit points to detect items moving into or out of storage areas, while additional readers can be placed within storage zones to associate items with logical locations [8, 9].

This deployment strategy enables zone-based tracking, where item location is inferred from the reader that detects the tag, rather than from precise positioning measurements. Such an approach reduces system complexity while still providing useful location awareness for inventory monitoring and management.

Chapter 3

Related Work

Contents

3.1	Case Study 1: Integration of RFID and CV for Autonomous Inventory Management	11
3.2	Case Study 2: Airport Smart Inventory and Baggage Management Using RFID	11
3.3	Case Study 3: Amazon Smart Inventory Management Using RFID and AWS	11
3.4	Research Study : Zone-Based RFID Tracking Approaches . .	12
3.5	Research Study : RFID-Based Asset Management in Controlled Environments	12
3.6	Comparative Analysis of Reviewed Systems	12

3.1 Case Study 1: Integration of RFID and CV for Autonomous Inventory Management

Decathlon, one of the largest sporting goods retailers, partnered with Simbe Robotics to develop an autonomous inventory tracking system using RFID and CV. The system utilizes the *Tally* robot, which is equipped with RFID antennas and high-resolution cameras to scan shelves and track inventory automatically. RFID technology enables fast and accurate identification of items, while CV verifies the physical placement of products on shelves.

The system operates across three main layers: a perception layer that collects data through RFID and cameras, a data processing layer where collected data is analyzed, and a management layer that presents results through real-time dashboards. This approach significantly improves inventory accuracy, reduces labor costs, and enables continuous monitoring, allowing faster detection of missing or misplaced items. The successful deployment of this system highlights the effectiveness of combining RFID with CV for scalable and autonomous inventory management in retail environments [13].

3.2 Case Study 2: Airport Smart Inventory and Baggage Management Using RFID

Hong Kong International Airport (HKIA) was among the first airports to adopt RFID technology for baggage and asset management. RFID tags are embedded into baggage and cargo units and are scanned by fixed RFID readers installed along baggage conveyors and sorting areas. This infrastructure enables real-time tracking of baggage movement throughout the airport.

The RFID-based system has significantly reduced the number of mishandled baggage incidents and improved overall operational efficiency. Reported benefits include a reduction in mishandled baggage by approximately 25–30% and an improvement in baggage processing time by up to 40%. Real-time dashboards provide operational transparency and enable airport staff to monitor asset movement efficiently. This case study demonstrates RFID’s effectiveness in large-scale, mission-critical environments and its applicability to inventory management systems that require high reliability and real-time visibility [14].

3.3 Case Study 3: Amazon Smart Inventory Management Using RFID and AWS

Amazon has integrated RFID technology into its inventory management operations to enhance real-time tracking and operational efficiency across its warehouses. RFID tags attached to items allow automated identification and tracking, eliminating the need for manual counting and reducing human error. Inventory data is continuously synchronized using cloud-based services.

The system leverages several AWS components, including AWS IoT Core for data collection, cloud-based databases for real-time storage, and serverless processing for handling inventory events. This architecture enables scalability across multiple warehouses and supports Amazon’s large and dynamic inventory operations. The integration of RFID

with cloud infrastructure demonstrates how inventory accuracy and processing speed can be improved through automation and real-time data synchronization [15, 16].

3.4 Research Study : Zone-Based RFID Tracking Approaches

Ni *et al.* introduced the LANDMARC system, an indoor location sensing approach based on active RFID technology that estimates object locations within indoor environments [17]. The system employs fixed reference tags and reader measurements to infer the approximate position of tagged objects without requiring precise distance calculations.

The study demonstrates that RFID reader detection patterns can be used to associate tags with logical regions, enabling location awareness with limited infrastructure. However, the system requires additional reference tags and focuses on coordinate-level localization, which increases deployment complexity.

3.5 Research Study : RFID-Based Asset Management in Controlled Environments

Wang *et al.* presented an RFID-based asset management system deployed in a hospital environment to track medical equipment and materials [18]. The system utilizes fixed RFID readers installed at strategic locations to automatically identify tagged assets and update inventory records as items move between different areas.

The proposed solution emphasizes zone-based tracking rather than precise localization, allowing assets to be associated with logical locations such as rooms or storage areas. Reported results demonstrate improved asset visibility, reduced manual tracking effort, and enhanced operational efficiency compared to traditional barcode-based systems. Although the application domain differs, the system architecture and tracking principles closely align with the proposed Smart RFID Inventory Tracking System, particularly in terms of reader-based allocation and deployment in controlled environments.

3.6 Comparative Analysis of Reviewed Systems

The reviewed systems demonstrate diverse approaches to achieving accuracy, scalability, and automation in inventory and asset management. Industrial deployments such as the Decathlon and Amazon systems rely on extensive RFID-based infrastructures to achieve high inventory accuracy and minimal human intervention. However, these solutions require significant investment in hardware, robotics, and cloud resources, leading to high deployment and operational costs.

Large-scale infrastructure-oriented solutions, such as the HKIA RFID system, employ fixed reader deployments optimized for controlled and mission-critical environments. While highly reliable, such systems are less adaptable to smaller or dynamically changing inventory settings.

Academic studies highlight alternative design philosophies. The LANDMARC system shows that RFID reader detection patterns can be used to infer object location, but its focus on coordinate-level localization and reliance on reference tags introduce additional

complexity. In contrast, the RFID-based hospital asset management system proposed by Wang *et al.* adopts a zone-based tracking approach within controlled environments, demonstrating that reliable asset visibility can be achieved without precise localization.

Compared to the reviewed systems, the proposed Smart RFID Inventory Tracking System adopts a balanced and practical design. It relies on RFID as the primary mechanism for tracking, counting, and allocation, while using computer vision only as an auxiliary data entry tool. This approach reduces system complexity and cost while maintaining accurate and up-to-date inventory records, making it well suited for laboratory and warehouse environments.

Table 3.1 summarizes the key differences between the reviewed systems and the proposed solution in terms of technology usage, automation level, infrastructure complexity, and application domain.

Table 3.1: Comparison between reviewed inventory management systems and the proposed system

Feature	Decathlon + Simbe Robotics	HKIA RFID System	Amazon RFID + AWS	Proposed System
Application Domain	Retail stores (shelf monitoring)	Airport baggage and assets	Large-scale warehouses	Laboratories and warehouses
Primary Tracking Technology	RFID	RFID	RFID	RFID
Real-Time Item Allocation	Yes (robot-based shelf mapping)	Yes (reader-based zones)	Yes (warehouse zone-based allocation)	Yes (reader-based zone allocation)
Use of CV	Yes (shelf verification)	No	No	Yes (auxiliary data entry only)
Automation Level	Fully autonomous (robot-based)	Semi-automated	Fully automated	Semi-automated (user-assisted)
Hardware Infrastructure	Autonomous robot with RFID and cameras	Fixed RFID readers	Fixed RFID readers with cloud infrastructure	Mobile device, RFID reader, Raspberry Pi
Cloud Integration	Limited	Limited	Extensive (AWS services)	Backend server with cloud-based AI services
Scalability	High (retail chains)	High (large airport)	Very high (global scale)	Medium to high
Cost	Very high	High	Very high	Low to medium
User Interaction	Minimal (autonomous operation)	Operational staff monitoring	Fully automated	User-assisted via mobile application and web dashboard
Target Objective	Real-time shelf accuracy	Baggage tracking and transparency	End-to-end inventory automation	Accurate, flexible, and cost-efficient inventory management

Chapter 4

System Design and Architecture

Contents

4.1 Overall System Architecture	16
4.2 System Layered Architecture	17
4.3 RFID Deployment Strategy and Range Control	19
4.3.1 Zone Readers vs. Door (In/Out) Reader	19
4.3.2 Read Range Control Methods	20
4.3.3 Approximate Coverage Estimation	20
4.3.4 Key Analytical Models (Supporting Equations)	20
4.3.5 RFID Interference and Mitigation Strategies	21
4.4 Hardware Components	22
4.4.1 Passive UHF RFID Tags	22
4.4.2 UHF RFID Reader Module	22
4.4.3 Raspberry Pi 4 (4GB RAM)	22
4.4.4 Mobile Device	23
4.5 Software Components	23
4.5.1 Technology Selection Rationale	24
4.6 System Requirements	25
4.6.1 Functional Requirements	25
4.6.2 Non-Functional Requirements	26
4.7 System Workflow	26
4.7.1 Item Entry into Warehouse	26
4.7.2 Zone Allocation and Tracking	27
4.7.3 New Item Registration	27
4.7.4 Item Exit from Warehouse	27
4.7.5 System Synchronization	28

The proposed Smart Inventory Management System is designed to improve inventory accuracy, reduce manual intervention, and maintain real-time synchronization between physical assets and digital records. The system uses passive UHF RFID technology as the primary mechanism for automatic item identification, tracking, and zone-level allocation within warehouse environments.

Passive RFID is selected due to its low cost, maintenance-free operation, and ability to support large-scale tagging without requiring line-of-sight. This makes it more suitable for warehouse inventory management than barcode systems, which depend on manual scanning and direct visibility.

To address the limitation of RFID in handling untagged or newly introduced items, the system integrates a computer vision (CV)-assisted data entry module. This module supports item registration only and does not replace the core RFID-based tracking mechanism. By separating tracking from registration assistance, the system maintains high reliability while reducing manual data entry effort.

The system adopts a zone-based allocation model instead of precise coordinate-level localization. In warehouse environments, identifying the reader zone in which an item is detected is sufficient for inventory management tasks such as presence detection, quantity tracking, movement logging, and allocation updates. Avoiding precise localization reduces system complexity and infrastructure cost while preserving operational effectiveness.

4.1 Overall System Architecture

The proposed Smart Inventory Management System follows a centralized architecture that integrates warehouse sensing devices, edge processing units, backend services, and user applications through a structured data flow. This design ensures consistent inventory information across all system components while simplifying coordination, monitoring, and data management.

At the sensing level, passive UHF RFID tags attached to inventory items are detected by fixed RFID readers deployed throughout the warehouse. The readers continuously monitor tag presence and generate identification events that reflect the location and movement of items.

To improve tracking reliability, the system employs two types of reader deployments: zone readers and gate readers. Zone readers are installed within storage areas and provide medium-range coverage (approximately 2–3 meters) to detect items located inside specific warehouse zones. Gate readers are installed at entry and exit points and are configured with a shorter reading range (approximately 0.5–1 meter). The shorter range of gate readers ensures that only items physically passing through doorways are detected, preventing false movement events caused by tags located near but not crossing the boundary. This separation between zone-level detection and transition-point detection improves movement logging accuracy and enhances inventory traceability while minimizing coverage overlap between readers.

Each reader is connected to a Raspberry Pi device, which serves as an edge processing unit. The Raspberry Pi handles communication with the reader hardware, structures tag detection events, timestamps the data, and forwards validated information to the backend server. Using an edge device reduces network load and isolates hardware-level operations from the core system logic.

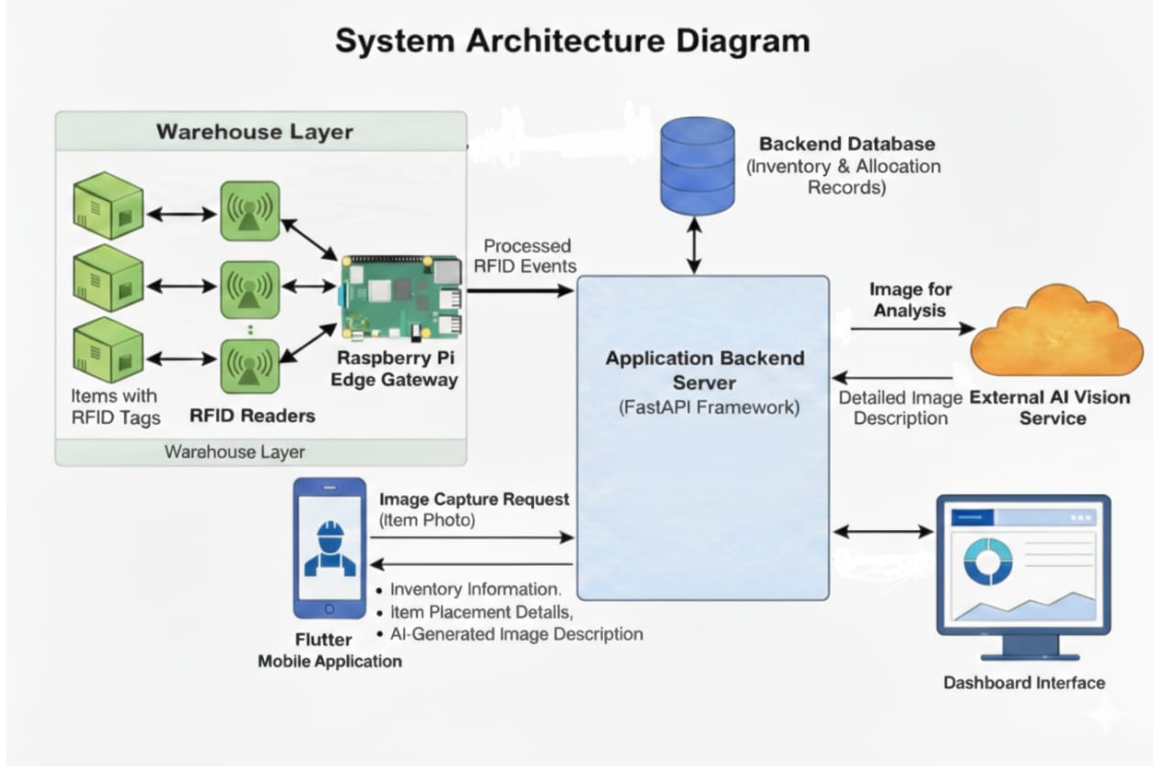


Figure 4.1: Overall system architecture of the proposed smart inventory management system.

The backend server, implemented using FastAPI, acts as the central control unit of the system. It manages inventory logic, zone allocation updates, movement logging, user authentication, and database operations. All inventory records are stored in a centralized PostgreSQL database to maintain data consistency across the system.

For item registration, the mobile application captures item images and sends them to the backend. The backend forwards these images to an external AI vision service (Gemini API) to obtain descriptive information. This description is compared with existing records to assist users in confirming whether the item is new or already registered. The vision module supports data entry only, while all tracking and allocation functions rely exclusively on RFID detections.

Mobile and web applications access the backend through RESTful APIs, providing real-time inventory visibility and management functionality without direct interaction with the hardware layer.

4.2 System Layered Architecture

To improve scalability, maintainability, and system organization, the proposed smart inventory management system adopts a layered architectural model. This approach separates physical sensing, edge processing, backend logic, and user interaction into distinct layers, allowing each part of the system to evolve independently without affecting the entire architecture.

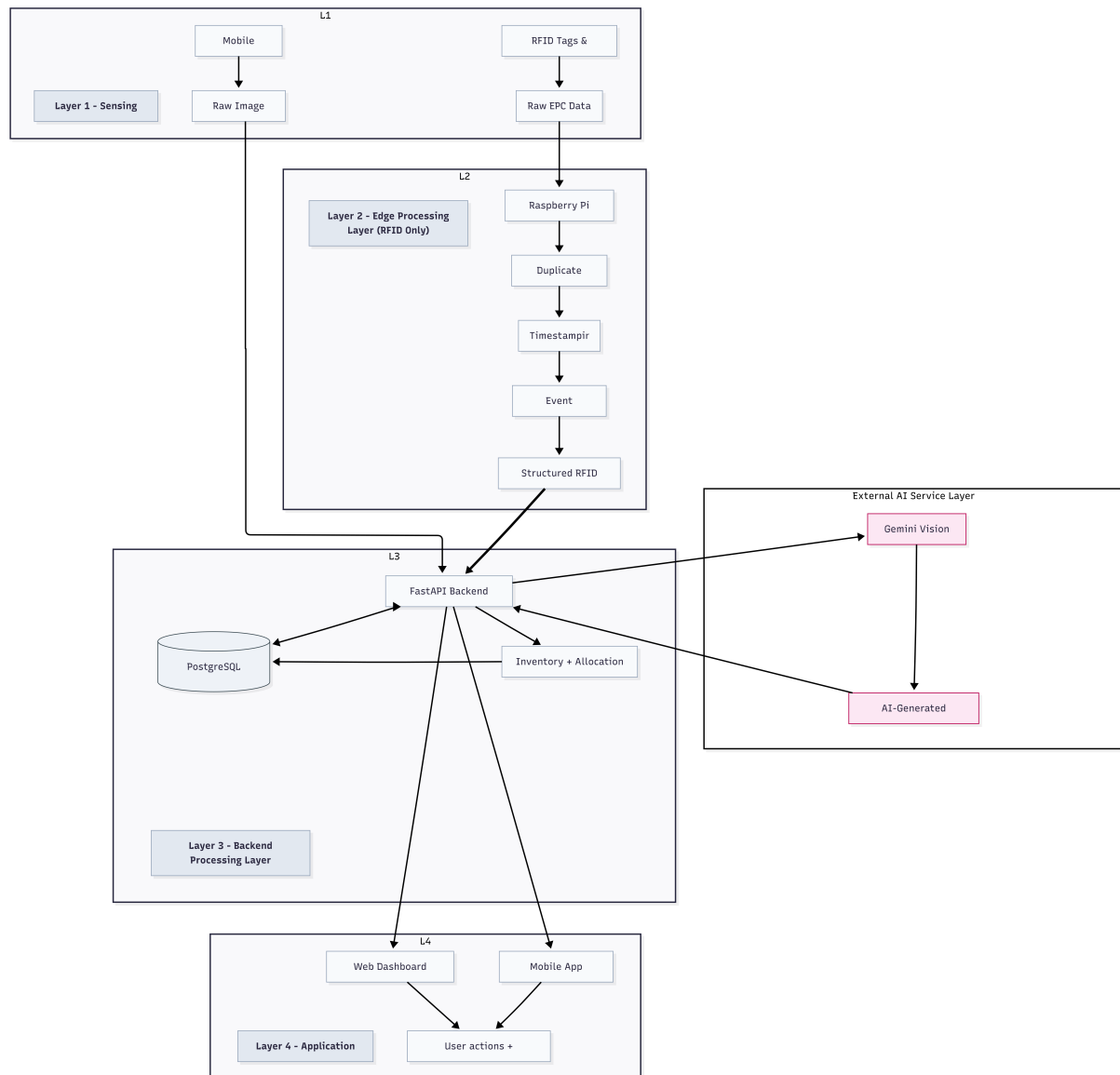


Figure 4.2: Layered architecture of the proposed system showing logical separation of system responsibilities.

Figure 4.2 presents the layered architecture of the proposed system, highlighting the logical separation of responsibilities and data flow between layers.

Layer 1 — Sensing Layer **Input:** RFID tag responses, captured item images. **Output:** Raw tag identifiers (EPCs), image data.

This layer interacts directly with the physical environment. Passive RFID tags reflect reader signals to transmit unique identifiers, while mobile device cameras capture images for new item registration. The purpose of this layer is raw data acquisition.

Layer 2 — Edge Processing Layer **Input:** Raw tag identifiers from RFID readers. **Output:** Structured detection events.

The Raspberry Pi operates at the edge, filtering duplicate tag reads, adding timestamps, and structuring detection data before transmission to the backend server. Local processing reduces communication overhead and improves responsiveness.

Layer 3 — Backend Processing Layer **Input:** RFID detection events, item images. **Output:** Inventory updates, allocation status, AI-generated descriptions.

The FastAPI backend performs system logic, database updates, zone allocation management, and communication with external AI services. This centralized layer ensures consistent decision-making and inventory synchronization.

Layer 4 — Application Layer **Input:** Inventory records and allocation data. **Output:** User actions and confirmations.

Mobile and web applications provide real-time system visibility. Workers use the mobile application for item registration and confirmations, while administrators use the web dashboard for monitoring and management.

4.3 RFID Deployment Strategy and Range Control

In practical inventory environments, the physical placement and configuration of RFID readers strongly influence system performance. The proposed system adopts a zone-based deployment strategy combined with a controlled transition reader.

4.3.1 Zone Readers vs. Door (In/Out) Reader

Two types of reader roles are defined in the system:

- **Zone Readers:** These readers are installed to cover specific storage areas such as shelves or storage zones. Their purpose is to detect the presence of tagged items within a zone to maintain allocation information and inventory quantities.
- **Door (Transition) Reader:** This reader is installed at entry/exit points of a warehouse or major storage area. Its purpose is to detect items moving into or out of the monitored environment. Unlike zone readers, this reader must operate with a smaller and more controlled detection region to avoid unintentionally reading tags from nearby zones.

4.3.2 Read Range Control Methods

Even when using the same hardware platform, the effective read range can be adjusted through configuration and deployment choices:

- Reducing the transmit power of the reader through software configuration.
- Adjusting reader orientation and mounting height.
- Introducing physical shielding or directional placement to limit unwanted reflections.
- Restricting reading time windows to moments when movement is expected.

4.3.3 Approximate Coverage Estimation

For planning purposes, if a zone reader achieves an effective read radius of approximately $r = 2$ meters in a given environment, the coverage area can be estimated as:

$$A = \pi r^2 = \pi(2)^2 \approx 12.6 \text{ m}^2$$

This estimation supports zone-based deployment planning. In real deployments, environmental factors such as metal surfaces, liquids, and interference may reduce this range, so calibration and on-site testing are required.

4.3.4 Key Analytical Models (Supporting Equations)

To support deployment planning and performance reasoning, a small set of simplified analytical models is used. These equations provide intuition for how range, coverage, and event rates scale with key parameters.

Friis-style free-space path loss (upper-bound range intuition):

$$P_r = P_t G_t G_r \left(\frac{\lambda}{4\pi R} \right)^2$$

where P_t is the transmit power, G_t and G_r are antenna gains, λ is the wavelength, and R is the separation distance. In practice, indoor multipath, polarization mismatch, and tag orientation cause additional losses, so the achievable read range is typically lower than the free-space estimate.

Reader coverage and density estimate:

If a single reader covers an effective area $A_r = \pi r^2$ and the total warehouse area is A_{tot} , an approximate lower bound on the number of readers required is:

$$N \geq \frac{A_{\text{tot}}}{\eta \pi r^2}$$

where $\eta \in (0, 1)$ is a coverage efficiency factor that accounts for overlap, obstructions, and non-uniform layouts.

Inventory/read-rate scaling with tag population:

Assuming the reader produces a usable identification event rate of R_{eff} (events/s), the expected time to observe N tags at least once scales as:

$$T_{\text{inv}} \approx \frac{N}{R_{\text{eff}}}$$

This simplified relationship captures the practical impact of higher tag populations on inventory latency.

Duplicate-read filtering model (time-window suppression):

To reduce redundant updates, the edge device can suppress repeated reads of the same EPC within a time window τ seconds. Modeling this as a non-paralyzable dead-time filter, the filtered event rate R_f can be approximated by:

$$R_f \approx \frac{R_{\text{raw}}}{1 + R_{\text{raw}} \tau}$$

where R_{raw} is the raw incoming read rate. This reduces database load while preserving movement-relevant events.

4.3.5 RFID Interference and Mitigation Strategies

The performance of UHF RFID systems in warehouse environments can be affected by environmental and deployment factors. The proposed system incorporates specific design choices to minimize interference and ensure reliable zone-based detection.

Metallic Surfaces: Metal reflects RF signals and may distort the electromagnetic field, leading to unstable or missed tag reads. *Mitigation in this project:* Items placed on metal shelves will use insulating spacers. Readers are installed with sufficient distance from large metal structures to prevent field distortion.

Liquid Materials: Liquids absorb RF energy, reducing effective read range. *Mitigation in this project:* For zones containing liquid-based items, readers are installed at shorter distances and configured using higher sensitivity settings to ensure reliable detection.

Tag Orientation: RF coupling efficiency depends on alignment between tag and reader antenna polarization. *Mitigation in this project:* The selected RFID reader modules include integrated circularly polarized antennas, allowing reliable detection regardless of tag orientation. Tag placement guidelines are defined to maintain consistent positioning.

Reader-to-Reader Interference: Overlapping RF fields from adjacent readers may cause cross-zone detections. *Mitigation in this project:* Reader placement is carefully planned so that coverage areas do not significantly overlap. Entrance readers are configured with shorter effective read ranges, while zone readers operate at moderate ranges. Transmission power settings are adjusted during installation to confine detection to the intended zone.

Multipath Effects: RF reflections from walls, shelves, and stored objects may create unstable signal patterns. *Mitigation in this project:* Readers are mounted at optimized angles and positions determined through on-site testing. Power levels are tuned during deployment to achieve stable coverage.

Through these deployment and configuration strategies, the system ensures stable tag detection and reliable zone-level allocation without requiring complex localization hardware.

4.4 Hardware Components

The hardware architecture of the proposed system is designed to balance cost, scalability, and integration flexibility while ensuring reliable RFID-based inventory tracking. The selected components are described below.

4.4.1 Passive UHF RFID Tags

Passive UHF RFID tags are attached to all inventory items. These tags store a unique EPC that enables automatic identification without requiring line-of-sight scanning.

Passive tags are selected instead of active tags for the following reasons:

- Low cost per unit, making them suitable for large inventories.
- No internal battery, eliminating maintenance requirements.
- Small size and flexible form factors (labels, cards, or hard tags).
- Compatibility with UHF readers that support multi-tag anti-collision.

4.4.2 UHF RFID Reader Module

The system uses a compact UHF RFID reader module compatible with Raspberry Pi via USB or Transistor-Transistor Logic (TTL) serial interface. The selected reader supports the ISO 18000-6C (EPC Gen2) standard and operates in the UHF band.

This type of module is chosen because:

- It provides sufficient read range (approximately 0–3 meters) for zone-level inventory tracking.
- It supports adjustable transmit power, allowing range control for door readers.
- It integrates easily with single-board computers.
- It offers a low-cost alternative to industrial fixed readers while remaining adequate for academic and small-to-medium deployments.

4.4.3 Raspberry Pi 4 (4GB RAM)

A Raspberry Pi 4 Model B with 4GB RAM is used as the edge processing unit between the RFID reader and the backend server.

The Raspberry Pi is selected for the following reasons:

- Provides sufficient processing power to handle RFID data filtering, buffering, and communication tasks.

- Supports Linux-based networking and Python environments for easy integration with backend Application Programming Interfaces (APIs).
- Offers USB, serial, and General Purpose Input/Output (GPIO) interfaces compatible with RFID modules.
- Low cost compared to industrial controllers.
- Large developer ecosystem and community support.

4.4.4 Mobile Device

A smartphone equipped with a camera is used by workers to interact with the system through the mobile application.

The mobile device enables:

- Image capture for CV-assisted item registration.
- User confirmation during inventory operations.
- Real-time visibility of inventory and zone allocation.

4.5 Software Components

The software architecture of the proposed system is designed to ensure scalability, real-time processing, and ease of integration between sensing devices and user interfaces. Each component was selected based on performance, development efficiency, and system maintainability.

- **Mobile Application – Flutter Framework** Flutter is used to develop the worker mobile application due to its cross-platform capabilities, allowing deployment on both Android and iOS devices from a single codebase. This reduces development time and ensures a consistent user interface. The mobile app supports item registration, confirmation of AI-generated suggestions, and real-time inventory visibility.
- **Backend Server – FastAPI Framework** FastAPI is selected for backend development because of its high performance, asynchronous request handling, and ease of building RESTful APIs. It supports real-time RFID event processing, database communication, and integration with external AI services. The framework also simplifies future system scaling.
- **Database System – PostgreSQL** PostgreSQL is chosen as the central database due to its reliability, support for structured relational data, and strong data integrity mechanisms. It efficiently stores inventory records, item descriptions, allocation status, and system logs.
- **AI Vision Integration – Google Gemini Vision API** Instead of developing a custom computer vision model, a cloud-based AI vision service is used. The Gemini Vision API provides object description capabilities that assist users in registering new items. This approach reduces computational requirements and development complexity while maintaining accurate image understanding.

4.5.1 Technology Selection Rationale

Table 4.1 summarizes the main technology choices made in this project and the rationale behind selecting each option based on cost, operational range, power considerations, and development time.

Table 4.1: Technology selection summary (cost, range, power, and development time).

Decision	Comparison (Cost / Range / Power / Dev time)	Chosen option (reason)
Edge processing unit	Raspberry Pi: higher unit cost, strong compute and Linux networking; suitable for continuous reader I/O and local filtering. ESP32: lower cost and low power; limited compute and more constrained networking/stack for this use case.	Raspberry Pi 4 (faster development and easier integration with RFID reader + backend APIs).
Mobile application framework	Flutter: single codebase for Android/iOS; shorter development time and consistent UI. Native (Swift/Kotlin): best platform integration, but longer development time (two codebases).	Flutter (reduced development time while meeting app requirements).
AI vision service	OpenAI vs. Gemini: both provide vision-capable models; selection driven by measured accuracy/override/time-saved results and integration constraints.	Gemini Vision API (integrated in the system design; OpenAI is evaluated as a benchmark in Chapter 5).
Identification technology	UHF RFID: meters-level range and multi-tag reading; suitable for warehouse zone tracking. NFC (HF): very short range (cm); typically requires close intentional tap/scan.	Passive UHF RFID (required for non-line-of-sight, zone-level, multi-item inventory tracking).

4.6 System Requirements

System requirements define the expected behavior, performance, and constraints of the proposed Smart RFID Inventory Tracking System. They are divided into Functional and Non-Functional requirements.

4.6.1 Functional Requirements

Table 4.2: Functional Requirements

ID	Requirement Description
FR1	The system shall automatically detect RFID tags when items enter or leave a reader zone.
FR2	The system shall update inventory quantities in real time based on RFID readings.
FR3	The system shall associate items with zones according to the reader that detects the tag.
FR4	The system shall record item movement history for traceability.
FR5	The backend shall send item images to the Gemini Vision API for description generation.
FR6	The system shall compare AI-generated descriptions with existing inventory records.
FR7	The user shall confirm whether the detected item matches an existing item or is new.
FR8	The system shall allow administrators to monitor inventory through a web dashboard.
FR9	The system shall maintain synchronized data between mobile devices, backend, and database.

4.6.2 Non-Functional Requirements

Table 4.3: Non-Functional Requirements

Category	Requirement
Performance	The system shall process RFID detection events with minimal delay to maintain near real-time inventory updates.
Scalability	The system shall support adding new RFID readers and zones without redesigning the architecture.
Reliability	Inventory data shall remain consistent even during temporary network interruptions.
Usability	The mobile application shall provide a simple and intuitive interface for workers.
Security	Administrative functions shall require user authentication and authorization.
Maintainability	The layered architecture shall allow independent updates of hardware, backend, and applications.
Cost Efficiency	The system shall rely on passive RFID technology and low-cost hardware components.
Accuracy	RFID-based tracking shall maintain high detection reliability within designed coverage zones.

4.7 System Workflow

The operational workflow of the proposed Smart Inventory Management System describes how items are registered, tracked, and updated within the warehouse environment. The workflow integrates RFID-based tracking with CV-assisted data entry while ensuring that all inventory updates remain synchronized with the backend database.

4.7.1 Item Entry into Warehouse

When a tagged item enters the warehouse through a controlled entry point, the **Door (In/Out) RFID Reader** detects the tag. This reader operates with a shorter and more focused read range to ensure that only items passing through the doorway are detected.

- The reader captures the tag's EPC identifier.
- The Raspberry Pi receives the detection event and timestamps it.
- The event is sent to the backend server.
- The backend updates the item status as *inside warehouse*.

This process enables automatic logging of item movement without manual scanning.

4.7.2 Zone Allocation and Tracking

After entry, items are placed in storage zones within the warehouse. Each zone is monitored by a dedicated **Zone RFID Reader**.

- Zone readers continuously scan for nearby tags.
- When a tag is detected, the backend associates the item with the corresponding zone.
- If the item is moved to another zone, the new reader detection updates its allocation automatically.

This ensures real-time allocation tracking at the zone level without requiring precise localization systems.

4.7.3 New Item Registration

If an item without an existing database record is introduced:

- The worker uses the mobile application to capture an image of the item.
- The image is sent to the backend server.
- The backend forwards the image to the external Artificial Intelligence (AI) Vision API.
- The API returns a generated description of the object.
- The backend compares this description with existing records.

Two outcomes are possible:

1. If a match is found, the worker confirms it and the quantity is updated.
2. If no match exists, the system assists in creating a new inventory record.

4.7.4 Item Exit from Warehouse

When an item leaves the warehouse:

- The Door (In/Out) Reader detects the tag.
- The backend logs the exit event.
- The system updates the item allocation and quantity records.

4.7.5 System Synchronization

All events are processed by the backend server, which ensures:

- Inventory quantities remain accurate
- Allocation data is updated in real time
- Mobile and web interfaces display consistent information
- Movement logs are stored for traceability

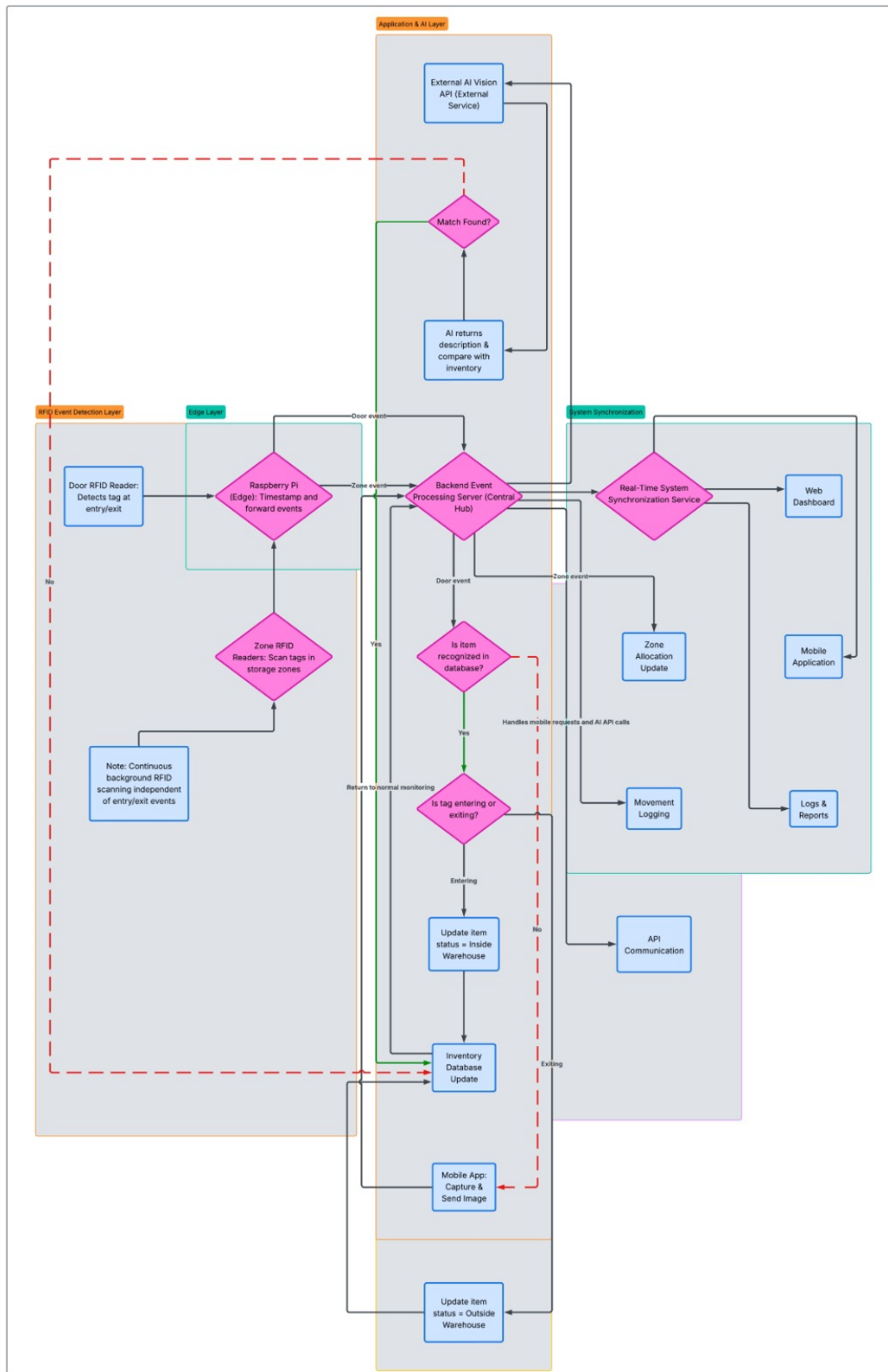


Figure 4.3: Operational workflow of the proposed RFID-based inventory system.

Chapter 5

System Implementation

Contents

5.1	Edge layer Implementation:	32
5.1.1	Raspberry Pi OS Installation and Initial Setup	32
5.1.2	UART Configuration on Raspberry Pi	33
5.1.3	Single RFID Reader Integration	33
5.1.4	Multi-Reader Setup Using USB-to-UART Adapter	34
5.1.5	RFID Data Collection Using Python	36
5.1.6	RSSI Extraction and Closest Reader Detection	36
5.1.7	Handling Reader Interference	37
5.1.8	Physical Testing with Different Materials	38
5.1.9	Preparing the JSON Payload	38
5.1.10	Sending Data from Raspberry Pi to Server	39
5.2	Backend Layer	39
5.2.1	Backend Architecture	39
5.2.2	Database Schema Design	41
5.2.3	API Design	42
5.2.4	RFID Detection Processing	43
5.2.5	Inventory and Order Management	46
5.2.6	Real-Time Communication	47
5.2.7	Mobile Application API	48
5.2.8	Error Handling and Validation	49
5.3	Dashboard Frontend Layer	50
5.3.1	Website	50
5.3.2	Mobile App	52
5.4	CV Integration Layer	54
5.5	Full System Integration and Validation	56
5.5.1	Full System Story Scenario	57
5.5.2	End-to-End Workflow Testing	58

5.5.3	RFID Scenario Testing	58
5.5.4	Cross-Layer Integration Verification	59
5.5.5	Validation Results	59

5.1 Edge layer Implementation:

In this system, the Raspberry Pi 4 works as the edge processing device. Its main responsibility is to connect with the RFID readers, collect RFID tag readings, process the received data locally, decide the most accurate reader for each detected tag, and prepare the final JSON payload that will be sent to the backend server.

The Raspberry Pi was selected because it provides enough processing power, supports Linux-based development, allows remote access through SSH, and provides hardware communication interfaces such as UART and USB, which are required for connecting RFID readers.

The Raspberry Pi side handles the following tasks:

- Reading RFID tag data from connected readers.
- Managing more than one RFID reader.
- Extracting tag IDs and RSSI values.
- Comparing RSSI values to estimate the closest reader.
- Filtering duplicate or repeated detections.
- Handling interference between readers.
- Preparing the JSON payload.
- Sending the processed data to the backend server.

5.1.1 Raspberry Pi OS Installation and Initial Setup

The implementation started by preparing the Raspberry Pi 4 environment. Raspberry Pi OS was installed on the device, and the system was configured to allow remote access through SSH. SSH access was important because it allowed development, testing, and debugging directly from a laptop without needing a separate monitor and keyboard for the Raspberry Pi.

After installing the operating system, the network connection was tested to make sure that the Raspberry Pi could be reached from the development machine. This step confirmed that the Raspberry Pi was ready for software development and hardware integration.

The main setup steps included:

- Installing Raspberry Pi OS.
- Connecting the Raspberry Pi to the network.
- Enabling SSH.
- Testing SSH access from the development machine.
- Updating the system packages.
- Preparing the Python environment for RFID testing.

5.1.2 UART Configuration on Raspberry Pi

After the basic Raspberry Pi setup, the required communication interfaces were configured. Since the RFID reader communicates using UART, the Raspberry Pi UART interface had to be enabled.

UART communication allows the Raspberry Pi to receive serial data directly from the RFID reader. This required enabling the serial interface and making sure that the correct Raspberry Pi pins were used for transmission and reception.

The UART setup was important because the RFID reader sends tag data as serial messages. Without enabling UART, the Raspberry Pi would not be able to receive data from the RFID reader.

The UART configuration included:

- Enabling the serial interface on Raspberry Pi.
- Making sure the UART port was visible and accessible.
- Connecting the reader TX pin to the Raspberry Pi RX pin.
- Connecting the common ground between the Raspberry Pi and the RFID reader.
- Testing the serial port using a simple Python script.

5.1.3 Single RFID Reader Integration

The first hardware integration step was connecting one RFID reader to the Raspberry Pi using UART. This was done to verify that the Raspberry Pi could receive data from the reader before moving to a multi-reader setup.

A simple Python script was developed to open the UART serial port and listen for incoming data from the RFID reader. When an RFID tag was placed within the reader's range, the reader sent the tag data to the Raspberry Pi.

During testing, the reading process was controlled through the Python Code so that the RFID reader reads and prints the detected tag data every 30 seconds. This controlled interval helped make the output more stable and easier to monitor during testing, while also reducing repeated readings and unnecessary processing.

The single-reader test helped verify:

- The RFID reader wiring was correct.
- UART communication was working.
- The Raspberry Pi could receive tag data.
- Python could read serial data from the reader.
- The tag ID could be extracted from the received message.
- The reader behavior and reading frequency were understood.

5.1.4 Multi-Reader Setup Using USB-to-UART Adapter

After successfully testing one RFID reader, the system was extended to support two RFID readers connected to the same Raspberry Pi.

To connect more than one RFID reader to the Raspberry Pi, three possible solutions were considered: using a USB-to-UART adapter, using a UART expander module, or using a UART multiplexer. The USB-to-UART adapter was selected for the prototype because it is simple, low-cost, and easy to integrate with Python serial communication. The UART expander module is a better option for future improvements because it provides a cleaner and more scalable hardware design. The UART multiplexer was not selected because it cannot read from all readers at the same time and may miss readings during the switching process.

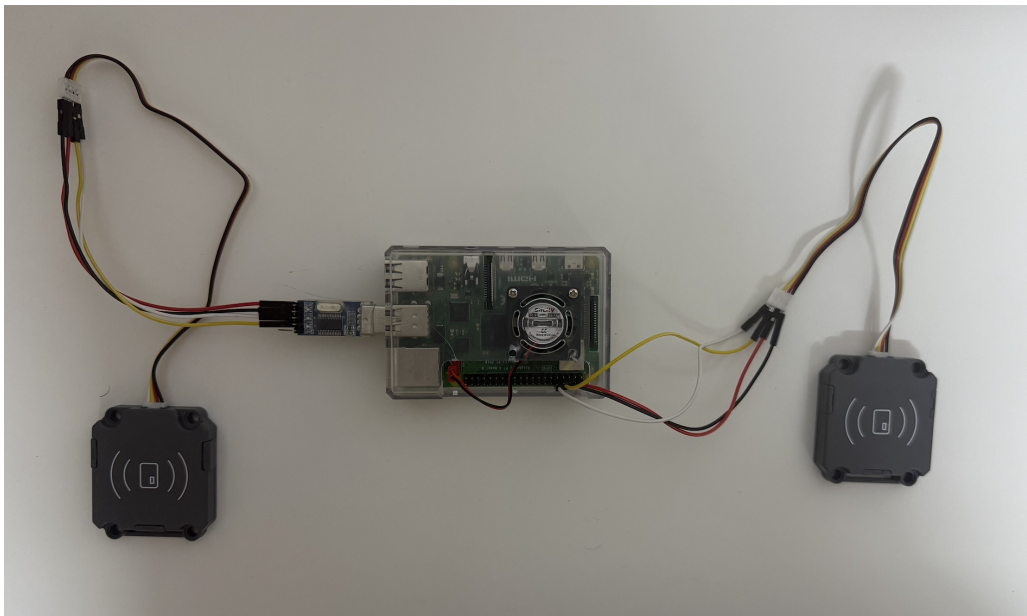


Figure 5.1: Raspberry Pi multi-reader connection setup.

Option	How it Works	Advantages	Disadvantages	Final Decision
USB-to-UART Adapter	Connects the second RFID reader to the Raspberry Pi through a USB port. The reader appears as a serial device such as <code>/dev/ttyUSB0</code> .	Simple, low-cost, easy to configure, works directly with Python serial communication, and supports reading from two readers at the same time.	Requires an external adapter, and the USB device name may change after reboot unless fixed with configuration.	Selected for our prototype because it was the fastest and most practical solution.
UART Expander Module	Adds extra UART ports to the Raspberry Pi using modules such as I2C/SPI UART expanders.	More professional and stable hardware design, supports multiple serial devices, and avoids depending on USB adapters.	Requires extra wiring, libraries, and driver setup. More complex than using USB-to-UART.	Good future improvement for a cleaner and more scalable design.
UART Multiplexer	Allows multiple RFID readers to share one Raspberry Pi UART port by switching between readers.	Cheap and uses only one UART port. Useful when simultaneous reading is not required.	Cannot read from all readers at the exact same time, requires switching logic, and may miss RFID readings during switching.	Not ideal for our system because reliable multi-reader detection is required.

Table 5.1: Comparison of possible multi-reader connection options.

Since the Raspberry Pi has limited built-in UART availability, a USB-to-UART adapter was used to connect the second RFID reader. This allowed the Raspberry Pi to communicate with two readers through two different serial interfaces:

- One reader connected through the Raspberry Pi UART port.
- Another reader connected through a USB-to-UART adapter.

This setup made it possible to read from both RFID readers at the same time. A Python script was developed to handle both serial ports and identify which reader sent each detection.

The multi-reader setup was important because the system needs to know not only which tag was detected, but also which reader detected it. This information is later used to estimate the location or closest reading zone of the tag.

The multi-reader implementation included:

- Connecting the first reader through Raspberry Pi UART.

- Connecting the second reader through USB-to-UART.
- Identifying the device path for each reader.
- Reading from both serial ports.
- Assigning a logical name to each reader.
- Testing both readers together.
- Verifying that each reader could detect tags independently.

5.1.5 RFID Data Collection Using Python

Python was used on the Raspberry Pi to collect and process RFID readings. The program continuously listens to the connected serial ports and waits for data from the RFID readers.

When a reader detects a tag, the Python program receives the raw data, extracts the required values, and adds metadata such as the reader name, reader ID, Raspberry Pi ID, and timestamp.

The collected data includes:

- Raspberry Pi ID.
- Reader name.
- Reader device path.
- RFID tag value.
- RSSI value.
- Estimated distance.
- Timestamp.

This local processing reduces the amount of unnecessary data sent to the backend server. Instead of sending raw reader output, the Raspberry Pi prepares clean and structured data.

5.1.6 RSSI Extraction and Closest Reader Detection

One of the important parts of the Raspberry Pi implementation was extracting the RSSI value from each RFID reading. RSSI represents the received signal strength between the RFID tag and the reader.

In this system, RSSI was used as an indicator to estimate which reader is closer to the detected tag. If the same tag is detected by more than one reader, the Raspberry Pi compares the RSSI values and selects the reader with the strongest signal as the closest reader.

This logic helps improve the accuracy of the system because RFID tags may be detected by multiple readers at the same time, especially when readers are physically close to each other.

The RSSI-based logic works as follows:

1. The same tag may be detected by Reader A and Reader B.
2. The Raspberry Pi collects the RSSI value from both readings.
3. The readings are compared.
4. The reader with the stronger RSSI value is selected.
5. The selected reader is considered the closest reader to the tag.
6. Only the most relevant detection is prepared to be sent to the server.

This approach helps reduce incorrect tag-location assignment.

5.1.7 Handling Reader Interference

During testing, it was noticed that RFID readers can interfere with each other, especially when they are placed close to each other or when the same tag is within the reading range of more than one reader. This created cases where the same tag could be detected by multiple readers at nearly the same time.

To handle this issue, the Raspberry Pi software uses a tag-to-reader mapping logic together with RSSI comparison. Instead of treating every raw reading as a final result, the software stores the current reader mapped to each detected tag and updates this mapping only when a stronger and more reliable reading is received from another reader.

This logic helps the system select the closest reader, reduce unstable switching between readers, and avoid sending repeated detections to the backend. For example, if a tag is already mapped to Reader A, another weak reading from Reader A is ignored. If the same tag becomes closer to Reader B and the RSSI value is stronger, the mapping is updated to Reader B.

The interference handling logic depends mainly on:

- Comparing RSSI values from different readers.
- Mapping each tag to the most accurate reader.
- Ignoring weak or repeated readings.
- Updating the mapped reader only when the new reading is more reliable.
- Sending only meaningful tag-location changes to the backend.

This made the edge-side logic more stable and suitable for real-world use.

5.1.8 Physical Testing with Different Materials

The RFID system was tested under different physical conditions to understand how the environment affects the reading accuracy.

Different materials and placements were tested, including:

- Normal tag placement.
- Tags near metal surfaces.
- Tags near water.
- Different distances between the tag and reader.
- Different reader positions.
- Different physical orientations of the tag.

The tests showed that materials such as metal and water can affect RFID reading performance. Metal can reflect or block RFID signals, while water can absorb part of the signal. Because of this, the physical placement of both the tag and the reader is very important.

Based on these tests, the reader setup was adjusted to improve reading stability and reduce false or unstable detections.

5.1.9 Preparing the JSON Payload

After collecting and processing RFID readings, the Raspberry Pi prepares the final data in JSON format before sending it to the backend server.

The agreed payload structure is an array of objects. Each object represents one RFID tag detection event. This structure allows the Raspberry Pi to send one or multiple detections in the same request.

Example payload:

```
[
  {
    "rpi_id": "pi-serial:1000000039cfdff9",
    "reader": "ReaderB_UART",
    "reader_id": "device:/dev/ttyS0",
    "reader_id_unique": false,
    "label": "TagA",
    "tag": "E280689400004031F0E0B5BEBE",
    "rssi": 192,
    "distance": "CLOSE",
    "timestamp": "2026-05-04T17:31:32Z"
  }
]
```

The payload fields are:

Field	Description
rpi_id	Unique identifier of the Raspberry Pi device.
reader	Logical name of the RFID reader.
reader_id	Device path or identifier of the reader.
reader_id_unique	Indicates whether the reader identifier is unique.
label	Optional label assigned to the detected tag.
tag	RFID tag EPC or unique tag value.
rsssi	Signal strength value received from the reader.
distance	Estimated distance category based on RSSI.
timestamp	Time of detection in UTC format.

Table 5.2: JSON payload fields sent by the Raspberry Pi.

5.1.10 Sending Data from Raspberry Pi to Server

Once the JSON payload is prepared, the Raspberry Pi sends it to the backend server through an HTTP request. This step connects the edge device with the server-side system.

The Raspberry Pi is responsible for sending only clean and meaningful detection data, while the backend is responsible for validating, storing, and displaying this data.

The communication process is:

1. RFID tag is detected by one or more readers.
2. Raspberry Pi reads the raw data.
3. Python script extracts tag and RSSI values.
4. Mapping and filtering logic is applied.
5. The closest reader is selected.
6. JSON payload is prepared.
7. Payload is sent to the backend server.
8. Server stores the detection in the database.

5.2 Backend Layer

5.2.1 Backend Architecture

The backend was implemented using FastAPI, a modern Python framework for building RESTful APIs. It acts as the central communication layer between the RFID hardware, PostgreSQL database, mobile application, and dashboard. The backend receives RFID detection events from Raspberry Pi devices, stores them in the database, updates item locations, manages inventory and orders, and sends real-time updates to connected clients through WebSocket.

The backend follows a layered architecture, as shown in Table 5.3.

Layer	Main Files or Directories	Responsibility
API Layer	app/routers/	Provides REST endpoints for items, orders, RFID detections, zones, employees, and real-time services.
Service / Business Logic Layer	app/tasks/	Handles RFID detection processing, item location updates, order management, item exit approval, and inventory quantity calculations.
Database Access Layer	app/models.py, app/database.py	Defines database tables using SQLAlchemy ORM and manages PostgreSQL connections and sessions.
Schema / Validation Layer	app/schemas.py	Validates incoming requests and formats outgoing responses using Pydantic schemas.
Real-Time Communication Layer	routers/realtime.py, websocket_manager.py	Broadcasts RFID detections and inventory updates to connected clients through WebSocket.
Migration Layer	alembic/versions/	Manages database schema changes safely as new tables, fields, and features are added.
Mobile and Dashboard API Layer	app/mobile_app/	Provides summarized data such as recent detections, latest orders, dashboard statistics, inventory maps, and pending RFID tags.

Table 5.3: Backend layered architecture.

FastAPI routers organize the endpoints into separate modules based on their function, which improves code organization and maintainability. The business logic is separated from the API routes and placed inside task modules. These modules handle the core system operations, including creating items, assigning RFID tags, processing detections, updating locations, creating orders, approving item exits, and recalculating inventory quantities.

The database layer uses SQLAlchemy ORM to communicate with PostgreSQL. Database tables are represented as Python classes in `app/models.py`, while connections and sessions are managed in `app/database.py`. Pydantic schemas are used to validate incoming data and control the structure of API responses, reducing invalid input and ensuring consistent communication between the backend and client applications.

The backend also supports real-time communication using WebSocket. When an RFID event is processed, connected clients receive updates immediately without repeatedly requesting new data. Alembic is used to manage database migrations and safely apply schema changes as the system develops.

In addition to the main CRUD endpoints, the backend provides aggregated endpoints for the mobile application and dashboard. These endpoints prepare summarized information, such as recent detections, order data, inventory analysis, and RFID pending tags, reducing the amount of processing required on the client side.

5.2.2 Database Schema Design

The backend uses a relational PostgreSQL database to store inventory information, RFID detection events, hardware devices, warehouse zones, orders, employees, approvals, and system settings. The schema connects physical RFID readings with actual inventory items and their current warehouse locations.

Main Tables

Table	Purpose	Main Columns
items	Stores general product and inventory information.	id, name, sku, cost_price, sold_price, total_qty, status, main_cat, sub_cat
variants	Represents item units or variants that can be tracked using RFID.	id, item_id, employee_id, rfid_tag_code, current_zone_id, qty
images	Stores image links associated with items.	id, item_id, url, spec, ev
zones	Represents warehouse locations such as shelves, storage areas, and gates.	id, name, zone_type, location_description
rpi_devices	Stores Raspberry Pi devices responsible for sending RFID readings.	id, device_id, name, status, last_seen_at
rfid_readers	Stores RFID readers connected to Raspberry Pi devices.	id, rpi_device_id, zone_id, name, reader_device_id, reader_type, status
rfid_detection_events	Records each RFID scan event and its related tracking information.	id, rpi_device_id, reader_id, zone_id, item_id, variant_id, tag, rssi, distance, detected_at, received_at
orders	Stores customer order information.	id, customer_name, customer_phone, customer_address, total_items, total_price, status
order_items	Stores the items or variants included in each order.	id, order_id, item_id, variant_id, quantity, qty_before, qty_after, approved_physically
approvals	Stores physical approval records for ordered variants leaving the warehouse.	id, order_id, variant_id, required_qty, v_qty_before, v_qty_after, p_approved, approved_date
employees	Stores employee accounts, roles, and access information.	id, name, email, password, role, phone, department, is_active
system_settings	Stores configurable operational values used by the system.	id, reader_poll_interval_seconds, gate_detection_window_minutes, missing_item_threshold_minutes

Table 5.4: Main backend database tables.

Table Relationships

One item can have multiple variants and images, while each variant belongs to one item. A variant may be assigned to a current warehouse zone, and each zone can contain multiple variants. A variant may also be assigned to an employee through `employee_id`, allowing the system to track which employee is responsible for or associated with a specific physical item unit.

A Raspberry Pi device can manage multiple RFID readers, while each RFID reader belongs to one Raspberry Pi device and may be assigned to a specific zone. Each RFID detection event may reference the related device, reader, zone, item, and variant.

One order can contain multiple order items, and each order item belongs to one order while referencing an item or a specific variant. An order can also have multiple approval records, where each approval links the order to a particular variant.

Database Operation

The database schema supports RFID-based inventory tracking by linking physical RFID tags to inventory variants and warehouse zones. Items are stored in the `items` table,

while their physical units are represented in the `variants` table. Each variant may have an RFID tag code and a current zone assignment.

RFID readers are connected to Raspberry Pi devices and may also be linked to warehouse zones. When a tag is detected, the event is recorded in the `rfid_detection_events` table together with the available device, reader, zone, item, and variant information.

Orders are stored separately and connected to the `order_items` and `approvals` tables. This structure allows the system to reserve inventory, record quantity changes, and verify the physical exit of ordered items from the warehouse.

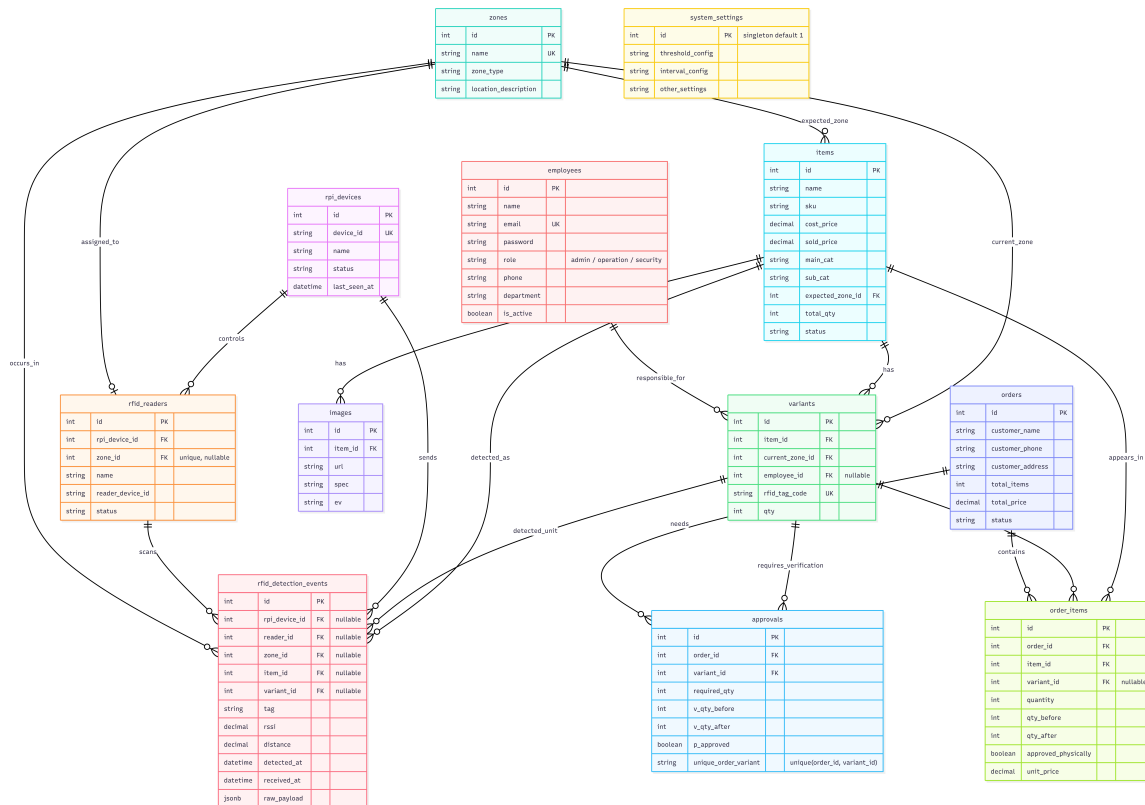


Figure 5.2: Backend database ER diagram.

5.2.3 API Design

The backend exposes a RESTful API using FastAPI. The API is divided into separate routers, where each router is responsible for a specific system domain, such as inventory, RFID detections, warehouse zones, devices, employees, orders, approvals, and system settings. This modular structure improves code organization and makes the backend easier to maintain, test, and extend.

Main API Groups

API Group	Purpose	Example Endpoints
Items API	Manages inventory items, variants, images, and RFID tag assignment.	GET /items, POST /items, PATCH /items/{item_id}/assign-rfid
RFID API	Receives RFID detections from Raspberry Pi devices and processes tag readings.	POST /rfid/detections
RFID Events API	Provides detection history, recent unknown tags, and item movement timelines.	GET /rfid-events/detections, GET /rfid-events/unknown-tags/recent
Zones API	Manages warehouse zones and provides zone summaries.	GET /zones, POST /zones, GET /zones/summary
RFID Readers API	Manages RFID readers and links them to Raspberry Pi devices and warehouse zones.	GET /rfid-readers, POST /rfid-readers
Raspberry Pi Devices API	Manages Raspberry Pi devices that send RFID readings to the backend.	GET /rpi-devices, POST /rpi-devices
Orders API	Manages customer orders and the items or variants included in each order.	GET /orders, POST /orders/with-variants, GET /orders/{order_id}
Approvals API	Manages physical approval records for ordered variants leaving the warehouse.	GET /approvals, PATCH /approvals/{approval_id}
Employees API	Manages employee accounts, roles, and authentication.	POST /employees/login, GET /employees
Settings API	Manages configurable system values and operational parameters.	GET /settings, PATCH /settings
Dashboard and Mobile API	Provides summarized data for dashboard and mobile application screens.	GET /dashboard/summary, GET /dashboard/recent-detections
Real-Time API	Provides live RFID events and inventory updates through WebSocket.	WS /ws/rfid-events

Table 5.5: Main backend API groups.

Request and Response Handling

The backend uses Pydantic schemas to validate incoming requests and format outgoing responses. This ensures that clients send the required fields in the correct format and receive consistent, structured data.

For example, when a Raspberry Pi device sends RFID detections, the request may include the following fields:

`rpi_id, reader, reader_id, tag, rssi, distance, timestamp`

The backend validates the request, processes each detection, associates it with the related reader, zone, item, and variant when possible, and stores the result in the database. It then returns a structured response indicating how many detections were received and stored, together with any unknown RFID tags that were identified.

Overall, the API design provides clear communication between the RFID hardware, back-end services, database, dashboard, and mobile application. The use of separate routers and validated schemas improves reliability and allows new endpoints and features to be added without affecting unrelated parts of the system.

5.2.4 RFID Detection Processing

RFID detection processing is one of the main backend operations. Raspberry Pi devices send RFID readings to the backend through the `POST /rfid/detections` endpoint. Each reading contains information about the Raspberry Pi device, RFID reader, detected tag, signal strength, estimated distance, and detection timestamp.

A detection request may contain the following fields:

`rpi_id`, `reader`, `reader_id`, `tag`, `rsssi`, `distance`, `timestamp`

Detection Processing Flow

When the backend receives RFID detections, it performs the following steps:

1. The Raspberry Pi sends one or more RFID detection records.
2. The backend identifies the Raspberry Pi device using `rpi_id`.
3. If the device is not registered, a new device record is created automatically.
4. The backend identifies the RFID reader connected to the device.
5. If the reader is not registered, it is also created automatically.
6. The system searches for a variant with the same RFID tag.
7. If the tag is registered, the detection is linked to the corresponding item and variant.
8. If the reader is assigned to a warehouse zone, the variant's current location is updated when appropriate.
9. If the tag is unknown, the detection is still recorded without item or variant references.
10. The event is stored in the `rfid_detection_events` table.
11. A real-time WebSocket message is broadcast to connected dashboard and mobile clients.

Known and Unknown RFID Tags

The system records both known and unknown RFID tags. A known tag is already assigned to an inventory variant, while an unknown tag has not yet been associated with any item.

Unknown tags are retained in the database so that employees can review them and later assign them to new or existing inventory variants through the mobile application.

RFID Detection Cases

The backend handles RFID detections according to the reader type, tag registration status, warehouse location, and order status.

Case	Situation	Backend Action
1. Normal movement between zones	A registered RFID tag is detected by a normal shelf or zone reader.	The backend stores the detection, updates the variant's <code>current_zone_id</code> , and broadcasts an item-movement update.
2. Registered warehouse entry	A registered tag is first detected by a gate reader and then by a normal reader within the configured time window.	The backend confirms the warehouse entry, increases the relevant variant quantity when required, updates its zone, and broadcasts an approved-entry event.
3. Unregistered warehouse entry	An unknown RFID tag is detected at the gate and later appears at an internal normal reader.	The backend records the event as an unregistered entry and sends a warning to the operations interface for review.
4. Ordered warehouse exit	A registered variant is detected at a gate reader and belongs to a pending customer order.	The backend approves the physical exit, updates the related order item, creates or updates the approval record, adjusts inventory, and may mark the order as collected.
5. Unauthorized warehouse exit	A registered variant is detected at a gate reader but does not belong to a pending order.	The backend records the event and generates a security warning because the item may be leaving the warehouse without authorization.

Table 5.6: RFID detection processing cases.

Gate Reader Logic

Gate readers are used to identify warehouse entry and exit events. When a registered item is detected at a gate, the backend checks whether the variant is associated with a pending order.

If the variant belongs to an active order, the backend can approve its physical exit and update the order and inventory records. If the variant does not belong to an order, the system generates a warning for possible unauthorized movement.

Entry events are identified by combining gate detections with later detections from internal readers. For example, when a tag is detected at a gate and then detected by a normal reader within the configured time window, the system treats the movement as a warehouse entry.

Overall, this processing logic allows the backend to track normal item movement, identify warehouse entry and exit events, process ordered items, preserve unknown RFID detections, and detect potentially unauthorized inventory movement.

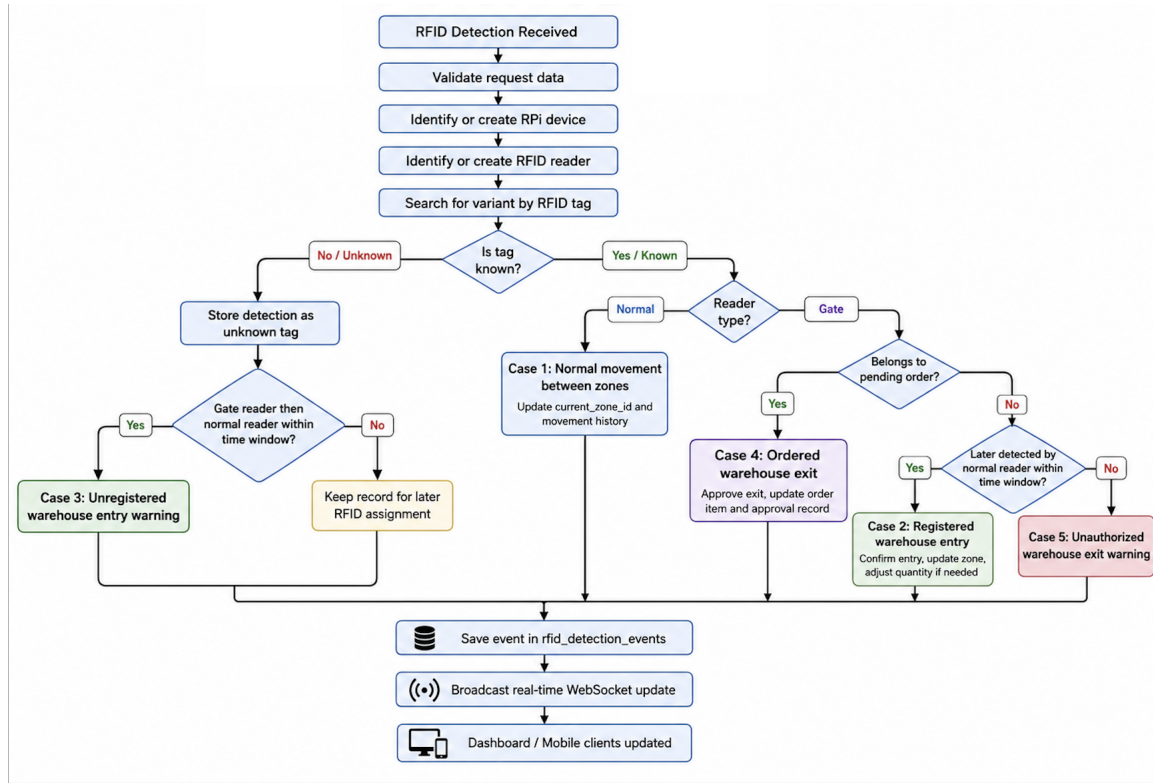


Figure 5.3: RFID detection processing flow.

5.2.5 Inventory and Order Management

The backend manages inventory using two main tables: `items` and `variants`. The `items` table stores general product information, while the `variants` table represents the physical units that can be tracked individually. Each variant may have an RFID tag, quantity, and current warehouse zone.

Inventory Management

The backend supports the main inventory operations required by the system, including:

- Creating, updating, deleting, and listing items.
- Adding variants to an item.
- Assigning or removing RFID tags.
- Storing item images.
- Tracking item quantities through variant quantities.
- Updating item locations after RFID detections.
- Recording unknown RFID tags for later assignment.

When a new item is created, it may include one or more variants and images. If no variants are provided but the item has an initial quantity, the backend can create a default variant automatically. The total quantity of an item is calculated from the combined quantities of its variants.

Order Management

Customer orders are stored in the `orders` table, while the selected products are stored in `order_items`. When an order is created, the backend validates the selected items and checks whether the requested quantities are available before accepting the order.

The main order operations include:

- Creating customer orders.
- Adding items or variants to an order.
- Validating available stock.
- Reserving or reducing variant quantities.
- Calculating the total number of items and total price.
- Updating or deleting orders.
- Tracking the physical approval status of ordered items.

Physical Approval Process

The `approvals` table is used to confirm that ordered items have physically left the warehouse. When an ordered variant is detected by a gate reader, the backend verifies that it belongs to a pending order. It then approves the exit, updates the related order item, creates or updates the approval record, and recalculates the order status.

The order status is updated according to the physical approval progress:

Status	Meaning
pending	No ordered items have been physically approved.
partial	Some, but not all, ordered items have been approved.
collected	All ordered items have been approved and collected.

Table 5.7: Order status based on physical approval progress.

Overall, this structure connects digital order processing with physical RFID-based verification. It ensures that inventory quantities, item locations, order records, and warehouse exits remain synchronized.

5.2.6 Real-Time Communication

The backend supports real-time communication using WebSocket. This allows connected dashboard and mobile clients to receive important RFID updates immediately, without continuously refreshing the page or repeatedly requesting new data from the server.

The main WebSocket endpoint is:

```
/ws/rfid-events
```

When a client connects to this endpoint, the backend registers the connection through a WebSocket connection manager. The manager keeps track of active clients and allows the backend to broadcast messages whenever an RFID event is processed.

Real-time notifications are generated for several important system events, including:

- Item movement between warehouse zones.
- Registered item entry into the warehouse.
- Unregistered item entry warnings.
- Physical approval of ordered items leaving the warehouse.
- Unauthorized exit security warnings.

5.2.7 Mobile Application API

The backend includes mobile-specific API endpoints that provide ready-to-use data for the mobile application. Instead of requiring the mobile app to call multiple endpoints and combine the results locally, the backend prepares summarized and structured responses. This reduces client-side processing and improves performance and consistency.

The mobile API supports dashboard summaries, warehouse zone information, recent RFID detections, latest orders, inventory analysis, unknown tag management, RFID mapping, image similarity checking, and AI-based part analysis.

Main Mobile Endpoints

Endpoint	Purpose
GET /dashboard/summary	Returns system totals, including items, variants, inventory quantity, active readers, online devices, pending orders, and unknown RFID detections.
GET /dashboard/zones	Returns warehouse zones together with the items and variants currently located in each zone.
GET /dashboard/recent-detections	Returns the latest RFID detection events for dashboard and mobile display.
GET /dashboard/latest-orders	Returns the most recent customer orders and their current statuses.
GET /mobile-app	Returns core system data, such as items, zones, RFID readers, Raspberry Pi devices, and system settings.
GET /mobile-app/analysis	Returns inventory value, expected sales value, potential profit, RFID assignment statistics, low-stock items, and category summaries.
GET /mobile-app/rfid/pending-tags	Returns recently detected RFID tags that have not yet been assigned to inventory variants.
POST /mobile-app/rfid/map-tags	Maps one or more pending RFID tags to an inventory item or variant.
GET /mobile-app/inventory-map	Returns tracked variants together with their RFID tags and current warehouse zone locations.
POST /mobile-app/similarity/check	Checks whether an uploaded part image is similar to an existing item image.
POST /mobile-app/gemini/analyze-part	Uses AI-based image analysis to identify a part and extract possible item information.

Table 5.8: Main mobile application API endpoints.

The dashboard endpoints provide summarized information for quick system monitoring, while the mobile endpoints support more detailed inventory operations. For example, employees can view unknown RFID tags, assign them to items, inspect variant locations, and analyze inventory values directly from the mobile application.

The backend also supports image-based inventory assistance. The similarity endpoint compares an uploaded part image with existing item images, while the AI analysis endpoint attempts to extract useful item information from the image. These features help employees identify parts and reduce manual data entry.

Overall, the mobile API layer simplifies communication between the backend and the mobile application by returning complete, structured responses that are ready for display and interaction.

5.2.8 Error Handling and Validation

The backend uses Pydantic schemas to validate incoming request data before it reaches the business logic. This ensures that required fields are included, data types are correct, and invalid values are rejected early.

Examples of validation rules include:

- Item names cannot be empty.
- Quantities must be greater than or equal to zero.
- Order quantities must be at least one.
- Employee passwords must meet the minimum length requirement.
- System setting values must remain within the allowed ranges.
- RFID detection requests must include fields such as `rpi_id`, `reader`, `reader_id`, `tag`, and `timestamp`.

FastAPI exception handling is also used to return clear and consistent error responses.

Common Error Responses

Status Code	Meaning	Example Situation
400 Bad Request	The request contains invalid data or represents an invalid operation.	Mapping RFID tags without providing an item ID.
401 Unauthorized	Authentication credentials are incorrect.	An employee enters an invalid email, username, phone number, or password.
403 Forbidden	The user is authenticated but not allowed to continue.	The employee account is inactive.
404 Not Found	The requested resource does not exist.	An item, order, reader, or zone cannot be found.
409 Conflict	The request conflicts with existing data.	An RFID tag is already assigned to another variant.
422 Unprocessable Entity	Request validation failed.	Required fields are missing or a field contains an invalid data type.
500 Internal Server Error	An unexpected backend error occurred.	An unhandled exception occurs during request processing.

Table 5.9: Common backend error responses.

Overall, this validation and error-handling approach prevents invalid data from entering the system and provides clients with clear information about failed requests.

5.3 Dashboard Frontend Layer

The frontend layer of the SmartRFID Inventory System is divided into two main interfaces: the website dashboard and the mobile application. Both interfaces communicate with the backend through REST API requests to retrieve and update stored data, and both use WebSocket connections to receive RFID events and system notifications in real time without requiring users to refresh the interface manually.

The frontend layer includes the following parts:

- **Website dashboard:** The website dashboard is built using React, TypeScript, Vite, Tailwind CSS, and reusable UI components. It provides the main browser-based interface for monitoring warehouse inventory, RFID readers, item movement, customer orders, employees, alerts, and system settings.
- **Mobile app:** The mobile application is built using Flutter and Dart. It provides a role-based interface for operation and security employees, allowing them to view dashboards, search items, manage orders, receive alerts, handle RFID-related events, and support inventory registration from a mobile device.

5.3.1 Website

Frontend Navigation

The sidebar contains the main navigation tabs of the SmartRFID system. Each tab represents a major function of the warehouse management interface.

Sidebar Tab	Function
Dashboard	Displays a general system overview, including total items, active RFID readers, recent item movements, pending orders, and security alerts.
Warehouse	Provides a visual warehouse view where items, zones, shelves, gates, and RFID readers can be monitored in real time.
RFID Events	Displays the history of RFID detection events, including detected items, readers, timestamps, signal information, and movement activity.
Movement Timeline	Tracks the movement history of a selected item or variant between different warehouse zones.
Items	Manages inventory items, variants, quantities, RFID tag assignments, item locations, images, and order creation.
Readers / Devices	Manages RFID readers and Raspberry Pi devices, including device status, reader type, connectivity, and zone assignment.
Orders	Handles customer orders, ordered items, order filtering, approval status, and order history.
Employees	Manages employee records, roles, contact details, account status, and employee creation or editing.
Alerts	Displays system warnings such as unauthorized exits, unknown item entries, missing items.
Settings	Manages system configuration values, including reader polling intervals, detection windows, alert thresholds, reconnect timing, and simulation speed.

Table 5.10: Frontend sidebar navigation tabs.

WebSocket Integration in the Warehouse View

The Warehouse page uses WebSocket communication to display item movements immediately after an RFID detection is processed by the backend. When the page is opened, the frontend establishes a connection to the WebSocket endpoint:

`/ws/rfid-events`

The connection remains active while the user is viewing the system. Whenever the back-end processes a new RFID event, it broadcasts a structured message to all connected clients. The Warehouse page receives this message and updates the visual interface without making another REST API request.

For example, when a variant moves from one shelf to another, the WebSocket message may include the RFID tag, item and variant information, previous zone, current zone, reader details, timestamp, and event type. The frontend then removes the item from its previous location and displays it in the new warehouse zone.

WebSocket events are also used to update the Warehouse page in the following situations:

- A registered item moves between internal warehouse zones.
- An item enters the warehouse through a gate.
- An unknown RFID tag enters the warehouse.
- An ordered item is approved to leave the warehouse.
- An item attempts to leave without a related order.
- A reader or Raspberry Pi device changes status.
- A security or operational warning is generated.

The frontend may use these events to update item cards, zone quantities, reader indicators, movement animations, alerts, and notification messages. For example, a normal movement can update the item location, while an unauthorized exit can display a warning message and visually highlight the related gate.

If the WebSocket connection is interrupted, the frontend can attempt to reconnect automatically. REST API requests remain useful for loading the initial warehouse state, while WebSocket is used to apply live changes after the page has loaded.

Overall, the combination of REST API and WebSocket provides an efficient communication model. REST endpoints load the current warehouse data, while WebSocket delivers real-time changes, allowing the Warehouse interface to remain synchronized with physical RFID activity.

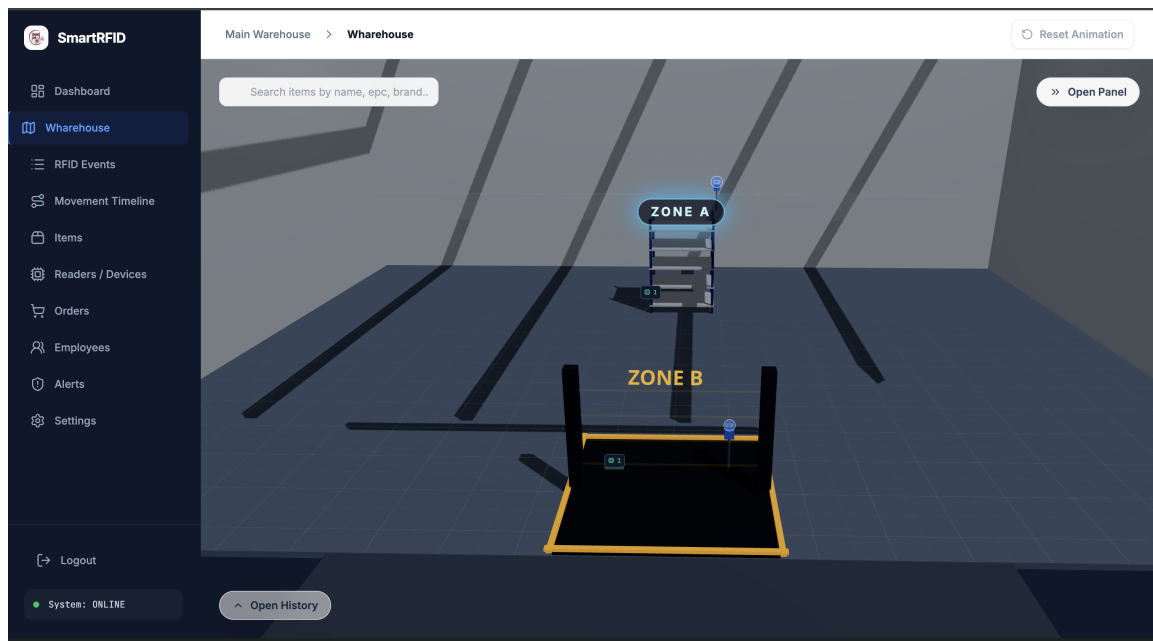


Figure 5.4: Warehouse view in the dashboard frontend.

5.3.2 Mobile App

The mobile frontend of the SmartRFID Inventory System was developed using Flutter and Dart. Flutter was selected because it supports Android and iOS applications using a single codebase. The application uses Material Design 3 components, including navigation bars, cards, forms, dialogs, status indicators, and notification panels, to provide a consistent and responsive interface.

The frontend is logically divided into three main parts:

- **Presentation components**, which display screens, forms, lists, alerts, dialogs, and navigation elements.
- **Data models**, which represent employees, inventory items, variants, orders, RFID movements, alerts, and dashboard summaries.
- **Service layer**, represented by `WarehouseService`, which communicates with the backend and converts JSON responses into frontend objects.

Authentication and Role-Based Interface

The application starts with a login screen where employees enter their username or email and password. The credentials are sent to the backend through the `POST /employees/login` endpoint. After successful authentication, the application displays navigation options according to the employee's assigned role.

Two main roles are supported:

Role	Available Functions
Operation Employee	Dashboard, orders, item search, operational alerts, inventory registration, and RFID tag assignment.
Security Employee	Security dashboard, item-location information, unauthorized-exit warnings, and security alerts.

Table 5.11: Mobile application roles and available functions.

Main Screens and Features

The application contains several screens that support warehouse and inventory operations:

Screen	Main Function
Dashboard	Displays inventory statistics, order information, recent alerts, and live RFID events.
Orders	Shows orders and their RFID-based completion status. Orders can be searched and filtered by incomplete, partially completed, collected, or all.
Find Item	Searches by item name, ID, SKU, variant ID, RFID tag, or warehouse zone. It also displays the selected item's current location and movement history.
Alerts	Displays missing items, misplaced items, unknown RFID tags, unregistered entries, unauthorized exits, and movement events. Alerts can be filtered, viewed, or resolved.
Data Entry	Allows operation employees to register new inventory items and assign pending RFID tags.
Camera and AI Entry	Captures or selects an image, compresses it, checks for visually similar items, and allows the employee to confirm an existing match or create a new item using AI-generated suggestions.
Security Dashboard	Prioritizes urgent security events, especially items detected at an exit without an approved order.

Table 5.12: Main Flutter mobile application screens.

Real-Time RFID Updates

The application connects to the `/ws/rfid-events` WebSocket endpoint to receive live RFID events. This allows item movements, entry events, exit approvals, and security warnings to appear without requiring the user to refresh the application manually.

WebSocket messages are filtered according to the active employee role. Operation employees may receive notifications about unknown or unregistered inventory entries, while security employees receive urgent warnings when an item attempts to leave without an approved order.

The application also supports Android local notifications for important background events. Unauthorized-exit events generate high-priority alarm notifications with sound, vibration, warning indicators, and a dialog directing security employees to the alerts screen. The alerts page refreshes periodically as a fallback in case the WebSocket connection is temporarily unavailable.

Frontend Validation and Testing

The frontend validates form input before sending data to the backend. Validation includes checking required fields, email and password values, item quantities, RFID assignments, and data-entry information. The interface provides clear error messages when input is missing or invalid.

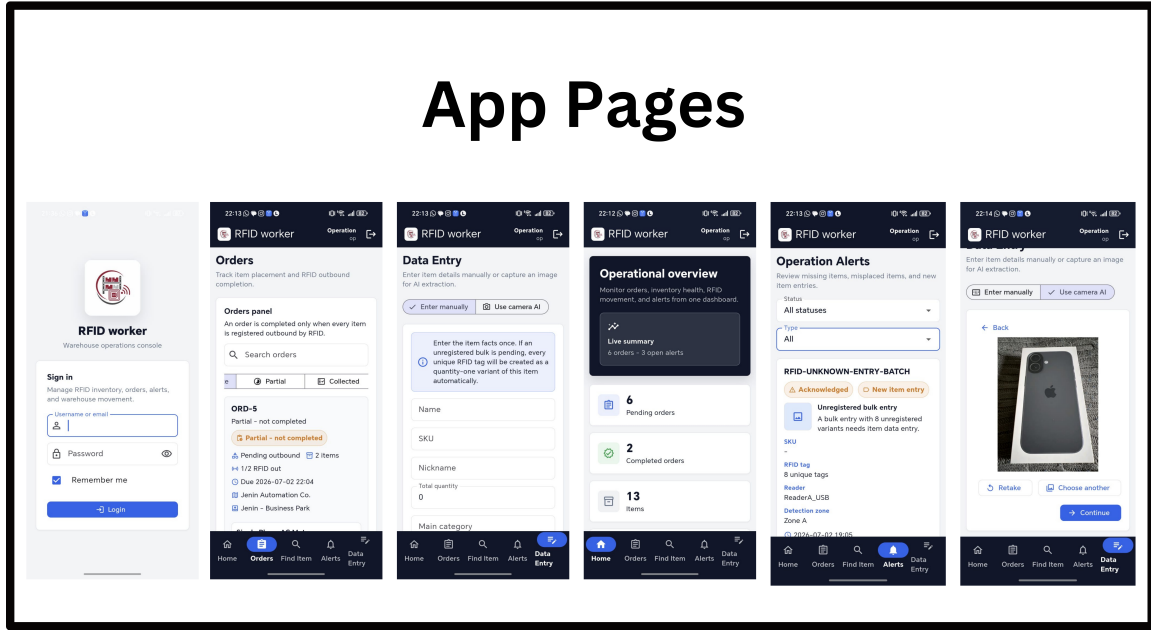


Figure 5.5: Main pages of the SmartRFID mobile application.

5.4 CV Integration Layer

The system includes a Computer Vision integration layer to support image-based inventory assistance. This layer helps the mobile application identify items using images, instead of depending only on manual search or RFID readings.

In our project, the CV feature is used when a worker uploads or captures an image of a part. The backend receives the image and compares it with the stored images of existing inventory items. Each item in the database can have a thumbnail or multiple related images, so the system uses these stored images as references during the comparison process.

When an image is uploaded, the backend first validates the image request and reads the image data. Then it collects the available item images from the database and prepares them for comparison. To compare images, the system converts the uploaded image and each stored image into a simple feature vector. This vector represents the image data in a numerical form, allowing the backend to compare images mathematically instead of comparing them as raw files only.

The feature vector is created from the image bytes by grouping byte values into buckets and normalizing them. This gives the system a compact representation of the image content. The backend also uses a sequence comparison between part of the uploaded image bytes and the stored image bytes. These two comparison results are combined to produce a final similarity score for each stored image.

The similarity score represents how close the uploaded image is to an existing item image. A higher score means the uploaded image is more likely to represent the same item. After calculating the scores, the system sorts the results and returns the most similar item to the mobile application.

To avoid weak or incorrect matches, the system uses a similarity threshold. In the current

implementation This means that the backend only considers the result a valid match if the best similarity score is equal to or greater than this value. If the score is below the threshold, the system does not assume that the item already exists in the inventory. Instead, it treats the result as not reliable enough and allows the image to be analyzed using Gemini.

This threshold is important because some items may look similar but still be different. For example, two products may have the same color or shape, but they may not be the same inventory item. The threshold helps reduce false matches and makes the system safer when updating inventory quantities.

This feature helps the worker quickly check whether a part is already registered in the system. For example, if the worker takes a picture of a product, the system can search the inventory images and return the closest matching item, including its name, image, quantity, and similarity score.

After the system suggests a match, the worker can confirm or reject the result. If the worker confirms that the suggested item is correct, the backend updates the inventory by increasing the quantity of that item. If the worker rejects the match, the system does not update the quantity and can send the image to Gemini for further AI-based analysis.

Gemini is used as an additional vision support tool. When the uploaded image does not match an existing item clearly, Gemini analyzes the image and returns structured information about the part, such as the part name, category, material, color, shape, visible features, possible use, condition, and confidence value.

The Gemini result is not applied automatically. Instead, the employee can review the returned information and decide whether to use it or not. If the employee accepts the Gemini result, the extracted data can help in creating or completing a new inventory item. If the employee rejects it, the system does not depend on the AI output. This keeps the final decision under human control and reduces the risk of adding incorrect item information.

The CV processing flow works as follows:

1. The worker uploads or captures a part image from the mobile application.
2. The backend receives and validates the image.
3. The backend collects the stored item images from the database.
4. The uploaded image and stored images are converted into feature vectors.
5. The backend calculates similarity scores between the uploaded image and each stored image.
6. The system compares the best score with the similarity threshold.
7. If the score passes the threshold, the system returns the most similar item as a reliable match.
8. The worker confirms or rejects the suggested match.
9. If the worker confirms the match, the item quantity is increased.

10. If the worker rejects the match, the image can be analyzed using Gemini.
11. Gemini returns structured information about the part.
12. The employee reviews the Gemini result and chooses whether to use the generated data or ignore it.

Overall, the CV integration layer improves the inventory system by adding visual item identification. RFID is used to track item movement and location, while the CV layer helps identify items from images, reduce manual searching, and support faster inventory registration. The use of feature vectors, similarity scoring, threshold-based matching, Gemini analysis, and employee confirmation makes the process more reliable and suitable for real warehouse use.

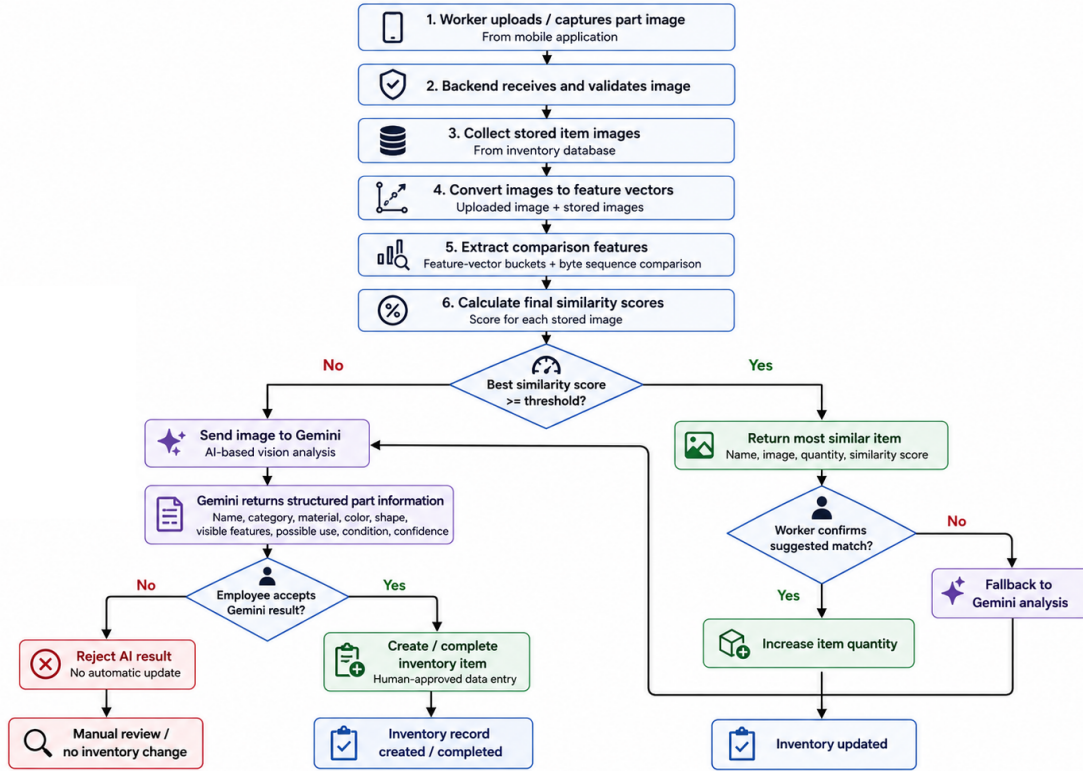


Figure 5.6: CV processing flow in the SmartRFID inventory system.

5.5 Full System Integration and Validation

After implementing the edge layer, backend layer, dashboard frontend, mobile application, and CV integration, full system integration testing was performed to verify that all modules operate together correctly. The goal of this section is to validate the complete workflow of the SmartRFID Inventory System, starting from RFID tag detection on the Raspberry Pi, sending data to the backend, processing and storing the event, updating inventory records, broadcasting real-time updates, and displaying the results on the dashboard and mobile application.

This validation confirms that the system layers are not isolated components, but work together as one connected solution for RFID-based inventory tracking.

5.5.1 Full System Story Scenario

The full system scenario starts when a new item arrives at the warehouse. The worker uses the mobile application to register the item by capturing its image. The CV module analyzes the image and helps extract item-related information. After confirming the item details, the worker attaches an RFID tag to the item.

When the item passes near an RFID reader, the Raspberry Pi collects the RFID tag, reader information, RSSI value, distance indicator, and timestamp. The edge layer prepares this information as a JSON payload and sends it to the backend API.

The backend receives the RFID detection, identifies the Raspberry Pi device and RFID reader, checks whether the RFID tag is registered, and links the detection to the correct item variant. If the reader is connected to a warehouse zone, the backend updates the item variant location. The detection event is then stored in the database and broadcast through WebSocket, allowing the dashboard and mobile application to update immediately.

If the detected RFID tag is not registered, the backend still stores the detection as an unknown-tag event and sends a real-time notification to the responsible operation employees. The unknown tag is then displayed in the pending RFID tags interface for review.

To simplify the identification process, the employee can use the Computer Vision feature by capturing or uploading an image of the detected item. The backend compares the image with stored inventory images and suggests the most similar registered item together with a similarity score. If the employee confirms the suggested match, the unknown RFID tag is linked to the corresponding item variant.

If no reliable match is found, the image can be analyzed using Gemini to extract possible item information, such as the item name, category, material, color, shape, and visible features. The employee reviews the generated information before creating a new inventory item and assigning the unknown RFID tag to it.

This integration combines RFID detection, image similarity, AI-assisted analysis, and human confirmation, ensuring that unknown items can be identified and registered efficiently without relying only on manual search.

Later, a customer order is created for the same item. The backend checks the available quantity, reserves the selected variant, calculates the order totals, and stores the order information. When the item leaves the warehouse and is detected by the gate reader, the backend verifies that the item belongs to a pending order. If it is valid, the system approves the physical exit, updates the order status, stores the approval record, and sends a real-time notification to connected clients. If it is not valid, such as when the item is not linked to an approved or pending order, the system rejects the exit, stores the detection as an unauthorized-exit event, generates a security warning, and broadcasts an urgent notification to the dashboard and mobile application.

The full story flow can be summarized as:

Item arrival → CV-assisted item registration → RFID tag assignment → RFID detection by Raspberry Pi → JSON payload sent to backend → Backend detection processing → Database update → Real-time dashboard/mobile update → Order creation → Gate detection → Exit approval

This scenario validates that the edge layer, backend layer, database, CV module, dashboard, and mobile application work together as one integrated system.

5.5.2 End-to-End Workflow Testing

End-to-end workflow testing was performed to verify that the system can complete a full inventory cycle from item registration to order approval. The test focused on confirming that data moves correctly between all layers of the system.

The tested workflow included:

1. Registering an item in the system.
2. Assigning an RFID tag to the item variant.
3. Detecting the RFID tag using an RFID reader connected to the Raspberry Pi.
4. Sending the detection payload to the backend.
5. Storing the detection event in the database.
6. Updating the item location based on the reader zone.
7. Displaying the updated information on the dashboard and mobile app.
8. Creating a customer order for the item.
9. Detecting the item at the gate reader.
10. Approving the item exit and updating the order status.

This test confirmed that the system can process data continuously across hardware, backend, database, and frontend layers.

5.5.3 RFID Scenario Testing

Several RFID scenarios were tested to verify that the backend correctly handles different real-world cases.

Scenario	Description	Expected Result
Registered item movement	A known RFID tag is detected by a normal zone reader.	The item variant location is updated to the detected zone.
Registered warehouse entry	A registered tag is detected at the gate and then inside the warehouse.	The system approves the entry, increases quantity, and updates the item zone.
Unregistered warehouse entry	An unknown tag is detected entering the warehouse.	The event is stored and an operation warning is generated.
Ordered item exit	A registered ordered item is detected at the gate.	The system approves the exit and updates the order approval status.
Unauthorized item exit	A registered item not linked to an order is detected at the gate.	The system generates a security warning.

Table 5.13: RFID scenario testing cases.

These scenarios verify that the system can distinguish between normal movement, valid entry, unknown item entry, approved exit, and unauthorized exit.

5.5.4 Cross-Layer Integration Verification

Cross-layer integration testing was used to verify that each system layer communicates correctly with the others.

Layer	Integration Verified
Edge layer	Raspberry Pi reads RFID tags and sends JSON payloads to the backend.
Backend layer	API receives payloads, applies business logic, updates the database, and stores inventory, RFID events, orders, approvals, zones, readers, and device records correctly.
Dashboard frontend	Receives WebSocket RFID events, displays inventory, orders, detections, and alerts from backend APIs, and synchronizes updated data across the dashboard and mobile application.
CV layer	Assists item registration by analyzing item images and supporting inventory creation.

Table 5.14: Cross-layer integration verification.

This verification ensures that the system components operate as a complete workflow rather than separate independent modules.

5.5.5 Validation Results

The validation results showed that the main system workflows operated correctly. RFID data was successfully collected by the Raspberry Pi, transmitted to the backend, processed according to the detection rules, stored in the database, and reflected in the dashboard and mobile application.

The tests confirmed the following results:

- RFID detections were successfully received and stored.

- Registered tags were linked to the correct item variants.
- Item locations were updated based on reader zones.
- Unknown tags were stored for later mapping.
- Order quantities and totals were updated correctly.
- Ordered item exits were approved through gate reader detection.
- Unauthorized exits generated security warnings.
- Real-time updates were broadcast to connected clients.
- Dashboard and mobile screens reflected updated backend data.
- CV-assisted item registration supported the inventory creation workflow.

Overall, the integration validation confirmed that the SmartRFID Inventory System works as a connected end-to-end solution. The system successfully combines RFID hardware, backend processing, database storage, real-time communication, dashboard visualization, mobile support, and CV-based item registration.

Chapter 6

Evaluation, Conclusion and Future Work

Contents

6.1	System Evaluation	62
6.2	Limitations and Issues	63
6.3	Conclusion	64
6.4	Future Work	65

6.1 System Evaluation

The SmartRFID Inventory System was evaluated by testing its main workflows across the hardware, backend, database, dashboard, mobile application, and Computer Vision layers. The evaluation focused on confirming that RFID detections were received correctly, backend logic processed events as expected, database records remained consistent, and frontend interfaces reflected system changes in real time.

The prototype was evaluated through a set of controlled warehouse scenarios. A total of **113 RFID detection events** were tested, including registered tags, unknown tags, normal item movements, warehouse entries, ordered exits, and unauthorized-exit cases. The system successfully detected, transmitted, and processed **108 out of 113 events**, resulting in an overall success rate of approximately **95.6%**, which can be rounded to **96%**. The five unsuccessful attempts were mainly caused by temporary RFID interference, unstable RSSI readings, tag orientation, and reader placement.

The evaluation covered the following main aspects:

- **RFID detection evaluation:** RFID readers connected to the Raspberry Pi were tested using both registered and unregistered tags. In the controlled test environment, **108 out of 113 tag-reading attempts** were successfully detected and transmitted to the backend, resulting in a detection success rate of approximately **95.6%**.
- **Backend processing evaluation:** The backend was tested using **60 workflow scenarios**, including normal item movement, registered warehouse entry, unknown-tag detection, ordered item exit, and unauthorized item exit. The backend produced the expected result in **60 out of 60 scenarios**, resulting in a backend scenario success rate of **100%**.
- **Database consistency evaluation:** Database records were inspected after each backend scenario to verify RFID detection events, item locations, variant quantities, order statuses, approval records, reader information, and generated alerts. The stored database state matched the expected result in **60 out of 60 scenarios**, resulting in a database consistency rate of **100%**.
- **Real-time update evaluation:** WebSocket communication was tested by monitoring the dashboard and mobile application after RFID events were processed. The average observed update delay was approximately **1 to 2 seconds**. A total of **56 out of 60 events** appeared automatically on connected clients without requiring a manual refresh, resulting in a real-time update success rate of approximately **93.3%**. The remaining events appeared after the frontend fallback refresh mechanism was triggered.
- **Frontend evaluation:** The dashboard and mobile application were tested through **60 interface workflow tests**, covering inventory display, item movement, order status, security alerts, unknown RFID tags, pending-tag mapping, and item-location information. The required data was displayed correctly on all major screens, although minor delays were observed during unstable network conditions.

- **CV-assisted registration evaluation:** The image-matching workflow was tested using **40 item images** captured under different lighting conditions, camera angles, and backgrounds. The system returned the correct or visually closest registered inventory item in **32 cases**, resulting in a preliminary matching accuracy of **80%**. In the remaining eight cases, the similarity result was below the accepted threshold, and Gemini-assisted analysis was used to suggest item information for employee review.
- **Human confirmation evaluation:** All image-matching and Gemini-generated results required employee confirmation before inventory data was created or updated. During the **40 CV tests**, no inventory quantity or item information was modified automatically without employee approval. This reduced the risk of incorrect registration caused by visually similar items or uncertain AI results.

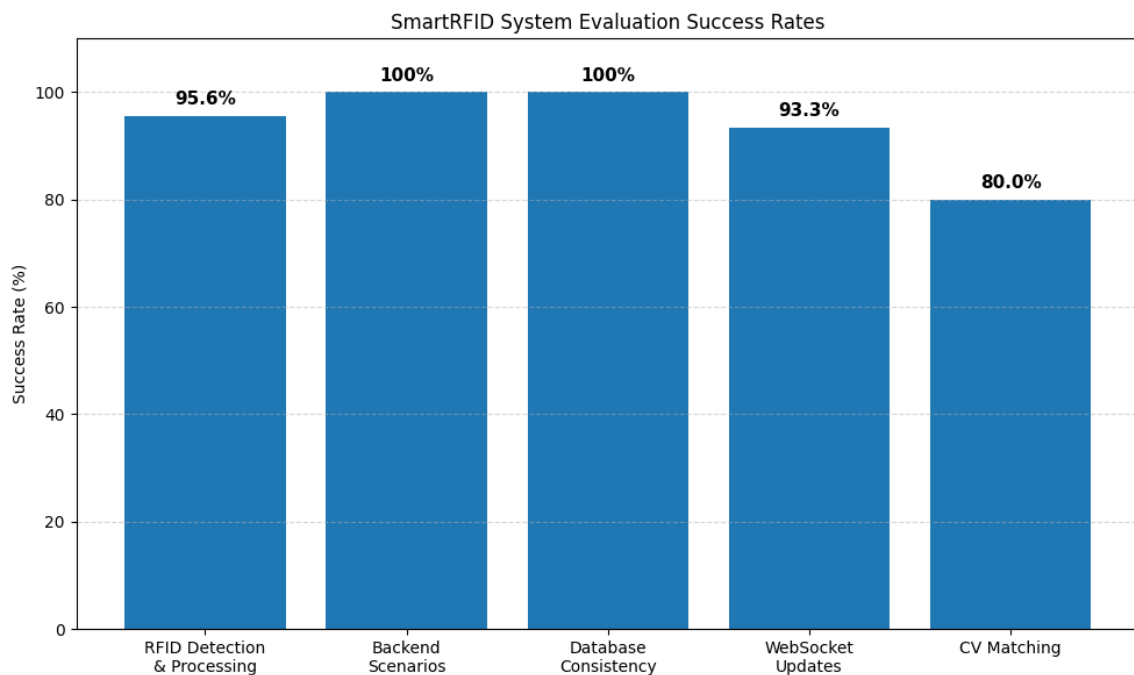


Figure 6.1: System evaluation results for the SmartRFID prototype.

Overall, the quantitative results demonstrate that the SmartRFID prototype can reliably integrate RFID hardware, Raspberry Pi edge processing, backend services, database storage, real-time communication, dashboard visualization, mobile operations, and CV-assisted inventory registration.

6.2 Limitations and Issues

Although the system achieved its main goals, several limitations and issues were identified during implementation and testing.

- **RFID interference:** RFID readers may interfere with each other when they are placed close together, especially when the same tag is detected by more than one reader at nearly the same time.

- **RSSI accuracy:** RSSI values provide a useful indication of the closest reader, but they are not always stable because signal strength can be affected by distance, obstacles, antenna direction, tag orientation, and environmental conditions.
- **Limited prototype scale:** The implemented prototype was tested with a limited number of readers, Raspberry Pi devices, items, and warehouse zones. Larger deployments may require more advanced device management and stronger performance optimization.
- **Network dependency:** Real-time updates depend on a stable network connection between the Raspberry Pi, backend server, dashboard, and mobile application.
- **High RFID event load:** A large number of repeated POST requests from RFID detection events may increase backend server load, cause higher response latency, or create temporary service outages if the server is not scaled or rate-limited properly.
- **CV matching reliability:** The CV layer supports item identification, but image similarity and AI-generated descriptions may not always be correct, especially when items look similar or the image quality is low.
- **Manual confirmation still required:** Human confirmation is still needed for item registration, AI results, unknown RFID mapping, and some order-related decisions to avoid incorrect inventory updates.

These limitations do not prevent the system from functioning, but they show the areas that should be improved before deploying the system in a larger real-world warehouse environment.

6.3 Conclusion

This project presented the design and implementation of the SmartRFID Inventory System, an integrated inventory-tracking solution that combines RFID hardware, Raspberry Pi edge processing, backend services, database storage, dashboard visualization, mobile support, and CV-assisted item registration. The system was designed to reduce manual inventory work, improve item-location tracking, and provide real-time visibility of warehouse activity.

The implemented system shows how RFID readers connected to a Raspberry Pi can detect tagged items, extract important information such as RFID tag values and RSSI readings, and send structured JSON payloads to the backend. The backend processes these detections, links registered tags to item variants, updates item locations, stores RFID events, manages orders, and broadcasts real-time updates through WebSocket. The dashboard and mobile application then display the updated warehouse state, alerts, orders, and item information to users.

The project also integrated a CV layer to support image-based item registration. This feature helps workers identify or describe items using images before assigning RFID tags, making the registration process easier and reducing the amount of manual data entry required. Together, the RFID and CV layers provide a more complete inventory-management workflow.

The system evaluation further confirmed the effectiveness of the prototype. In controlled testing, the system successfully detected, transmitted, and processed 108 out of 113 RFID events, achieving an approximate success rate of 95.6%. Backend processing and database consistency achieved the expected results in all tested workflow scenarios, while Web-Socket updates allowed most RFID events to appear automatically on the dashboard and mobile application within about one to two seconds. The CV-assisted registration workflow also supported inventory creation by returning correct or visually close matches in most image tests, while still requiring employee confirmation before updating inventory data.

Overall, the system demonstrates that a low-cost hardware setup, combined with modern backend and frontend technologies, can provide an effective smart inventory solution. The integration validation confirmed that the main workflows operate as a connected end-to-end system, from item registration and RFID detection to database updates, real-time notifications, order approval, and frontend visualization.

6.4 Future Work

Several improvements can be added in future versions of the SmartRFID Inventory System.

- **Expand the RFID deployment:** Add more RFID readers, Raspberry Pi devices, gates, shelves, and warehouse zones to test the system under larger operating conditions.
- **Improve reader synchronization:** Add more advanced scheduling or anti-collision techniques to reduce reader interference and improve detection reliability.
- **Use dedicated UART expansion hardware:** Replace USB-to-UART adapters with UART expander modules for a cleaner and more scalable hardware design.
- **Enhance localization accuracy:** Combine RSSI with additional techniques such as reader calibration, antenna tuning, signal averaging, or zone-based filtering to improve closest-reader detection.
- **Improve CV recognition:** Train or integrate a more specialized image-recognition model for warehouse parts and compare its accuracy against the current image-similarity and Gemini-based workflow.
- **Improve security and access control:** Add stronger authentication, permission management, audit logs, and deployment-level security settings.
- **Support offline operation:** Allow the Raspberry Pi or mobile application to temporarily store events locally when the network is unavailable and synchronize them when the connection returns.

Future work should focus on improving reliability, scalability, accuracy, and usability so that the system can move from a working prototype to a more production-ready smart warehouse solution.

Bibliography

- [1] M. Attaran, “Rfid: An enabler of supply chain operations,” *Supply Chain Management: An International Journal*, vol. 12, no. 4, pp. 249–257, 2007. Accessed: December 11, 2025.
- [2] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards and Identification*. John Wiley & Sons, 3rd ed., 2010. Accessed: December 13, 2025.
- [3] A. Juels, “Rfid security and privacy: A research survey,” *IEEE Journal on Selected Areas in Communications*, vol. 24, no. 2, pp. 381–394, 2006. Accessed: December 13, 2025.
- [4] D. M. Dobkin, “The rf in rfid: Passive uhf rfid in practice,” *IEEE Communications Magazine*, vol. 46, no. 9, pp. 32–41, 2008. Accessed: December 13, 2025.
- [5] P. V. Nikitin and K. V. S. Rao, “Theory and measurement of backscattering from rfid tags,” *IEEE Antennas and Propagation Magazine*, vol. 48, no. 6, pp. 212–218, 2006. Accessed: December 13, 2025.
- [6] ISO/IEC, “Iso/iec 18000: Radio frequency identification for item management,” 2019. Accessed: December 13, 2025.
- [7] S. Preradovic and N. C. Karmakar, “Chipless rfid: Bar code of the future,” *IEEE Microwave Magazine*, vol. 11, no. 7, pp. 87–97, 2010. Accessed: December 13, 2025.
- [8] L. Ruiz-Garcia and L. Lunadei, “The role of rfid in agriculture: Applications, limitations and challenges,” *Computers and Electronics in Agriculture*, vol. 79, no. 1, pp. 42–50, 2011. Accessed: 16 December 2025.
- [9] A. H. Sayed and M. N. El-Gamal, “Rfid-based localization and tracking in smart warehouses,” *IEEE Communications Magazine*, vol. 55, no. 1, pp. 26–32, 2017. Accessed: 16 December 2025.
- [10] S. Sarac, N. Absi, and S. Dauzère-Pérès, “A literature review on the impact of rfid technologies on supply chain management,” *International Journal of Production Economics*, vol. 128, no. 1, pp. 77–95, 2010. Accessed: 16 December 2025.
- [11] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2011. Accessed: December 13, 2025.
- [12] M. Leo and T. De Paolis, “Computer vision for automated inventory management,” *IEEE Access*, vol. 7, pp. 152558–152568, 2019. Accessed: December 13, 2025.

- [13] Avery Dennison RFID, “How rfid enables autonomous robot-based inventory management at decathlon,” 2025. Accessed: November 9, 2025.
- [14] Lyngsoe Systems, “Rfid solutions for airports and airlines,” 2024. Accessed: November 9, 2025.
- [15] AWS Industries Blog, “Enhancing retail store inventory management through rfid technology on aws,” 2023. Accessed: November 9, 2025.
- [16] RFID Label, “How does amazon use rfid in logistics?,” 2024. Accessed: November 9, 2025.
- [17] L. M. Ni, Y. Liu, Y. C. Lau, and A. P. Patil, “Landmarc: Indoor location sensing using active rfid,” in *Proceedings of the First IEEE International Conference on Pervasive Computing and Communications (PerCom)*, IEEE, 2003. Accessed: 29 December 2025.
- [18] S. W. Wang, W. H. Chen, C. S. Ong, L. Liu, and Y. W. Chuang, “Rfid application in hospitals: A case study on a demonstration rfid project,” in *Proceedings of the 39th Annual Hawaii International Conference on System Sciences (HICSS)*, IEEE, 2006. Accessed: 29 December 2025.